

TUGAS BESAR I

IF3170 INTELEGENSI ARTIFISIAL

Disusun oleh:

Tengku Naufal Saqib (13524012)

Rhenaldy Cahyadi Putra (13524039)

Fauzan Mohamad Abdul Ghani (13524113)

Daftar Isi

BAB I | Deskripsi Persoalan **3**
1.1 Variabel Permasalahan 3
1.2 Ketentuan Representasi 3
1.3 Fungsi Objektif dan Batasan 4
BAB II | Pembahasan **5**
2.1 Gambaran Umum Implementasi 5
2.2 Representasi State 6
2.3 Fungsi Objektif 6
2.4 Fungsi Neighbor 7
2.5 Evaluasi Neighbor Inkremental 8
2.6 Implementasi Algoritma Local Search 8
2.6.1 Steepest Ascent Hill-climbing 9
2.6.2 Hill-climbing with Sideways Move 9
2.6.3 Random Restart Hill-climbing 9
2.6.4 Stochastic Hill-climbing 9
2.6.5 Simulated Annealing 9
2.6.6 Genetic Algorithm 10
2.7 Deskripsi Kelas dan Fungsi 11
2.8 Visualisasi 12
2.9 Source Code 13
2.10 Hasil Eksperimen 35
2.10.1 Lingkungan, Data Uji, dan Parameter 35
2.10.2 Steepest Ascent HC 36
2.10.3 HC with Sideways Move 37
2.10.4 Random Restart HC 39
2.10.5 Stochastic HC 40
2.10.6 Simulated Annealing 42
2.10.7 Genetic Algorithm 44
2.10.8 Eksperimen Tambahan 46
2.11 Analisis Hasil Eksperimen 47
2.11.1 Kedekatan terhadap Global Optimum 47
2.11.2 Pola Pencarian 48
2.11.3 Perbandingan Varian Hill-climbing 48
2.11.4 Simulated Annealing 49

2.11.5 Genetic Algorithm..... 50
2.11.6 Durasi dan Konsistensi..... 51
2.11.7 Mode Objective Density.....51
BAB III | Kesimpulan dan Saran..... 52
3.1 Kesimpulan.....52
3.2 Saran..... 52
Lampiran..... 53
Link Repository.....53
Lampiran Penggunaan AI.....53
Pembagian Tugas Kelompok.....53
Referensi.....53

BAB I | Deskripsi Persoalan

Keluarga Mejiro sedang menguji coba usaha logistik darat bernama Mejiro Express. Sebagai pengemudi uji coba, Special Week membutuhkan program yang dapat menentukan konfigurasi pengisian paket ke dalam sebuah peti kemas truk secara otomatis. Tujuannya adalah memaksimalkan total value paket yang diangkut, tanpa melanggar batasan fisik peti kemas.

Secara formal, persoalan ini merupakan gabungan dari 0/1 knapsack problem (memilih paket mana yang diangkut agar total value maksimum dengan batas berat) dan three-dimensional container loading problem (menentukan posisi dan orientasi setiap paket di dalam ruang tiga dimensi). Persoalan ini bersifat NP-hard sehingga pencarian solusi eksak tidak praktis untuk instance berukuran sedang. Oleh karena itu, persoalan diselesaikan menggunakan algoritma local search.

1.1 Variabel Permasalahan

Entitas	Atribut	Contoh	Keterangan
Package	id	"RusdiPackage"	Identifier paket
	Dimensions	{w:3, l:4, h:1}	Ukuran paket: width, length, height
	Orientation	{w:x, l:y, h:z}	Sumbu yang sejajar dengan tiap sisi paket (6 kemungkinan)
	Value	60	Nilai harga paket (satuan unit)
	Weight	5	Berat paket (satuan unit berat)
	IsFragile	False	Penanda benda pecah-belah
	ETA	3	Perkiraan waktu kedatangan (sisa hari)
Truck	Dimensions	{w:10, l:20, h:5}	Ukuran peti kemas
	MaxCapacity	500	Kapasitas angkut maksimum (satuan unit berat)

Tabel 1.1 Variabel permasalahan

1.2 Ketentuan Representasi

- Paket berbentuk balok. Tidak semua paket harus dimasukkan; paket dapat berada seluruhnya di luar peti kemas.
- Koordinat berupa bilangan bulat. State awal dibangkitkan secara acak.

- Langkah neighbor yang diperbolehkan: (1) menukar dua paket, baik di dalam maupun di luar truk; (2) memindahkan sebuah paket ke koordinat yang berbeda; (3) merotasi paket 90° terhadap sumbu x, y, atau z.
- State akhir harus menunjukkan keadaan semua paket (di dalam maupun di luar), keadaan peti kemas, dan value state tersebut.

1.3 Fungsi Objektif dan Batasan

Fungsi objektif adalah total value paket yang berada di dalam peti kemas, yang ingin dimaksimalkan. Batasan yang harus dipenuhi:

1. **Tidak menembus dinding**. Paket di dalam truk harus seluruhnya berada di dalam batas peti kemas.
2. **Tidak saling tumpang-tindih**. Dua paket tidak boleh menempati ruang yang sama.
3. **Tidak melayang**. Paket dianggap melayang jika tidak ada satu pun titik pada alasnya yang memiliki support, yaitu berada di lantai (z = 0) atau tepat di atas permukaan atas paket lain.
4. **Paket pecah-belah tidak boleh menjadi support** bagi paket lain.
5. **Total berat** paket di dalam truk tidak boleh melebihi kapasitas maksimum.

BAB II | Pembahasan

2.1 Gambaran Umum Implementasi

Program diimplementasikan dalam bahasa Python 3.11 tanpa library external untuk local search. Visualisasi menggunakan library matplotlib. Struktur implementasi program adalah sebagai berikut.

File	Isi
src/core/models.py	Kelas paket dan truk, fungsi rotate paket dan daftar 6 rotasi dari paket.
src/core/state.py	Kelas placement dan state, pembangkitan state awal acak.
src/core/neighbors.py	Definisi Move, penerapan move, enumerasi dan pemilihan acak neighbor.
src/core/objective.py	Perhitungan fungsi objektif dan pelanggaran terhadap constraint
src/core/evaluator.py	Menghitung fungsi objektif neighbor
src/core/io_utils.py	Membaca input JSON dan menyimpan hasil run ke folder output.
src/algorithms/base.py	Kelas abstrak LocalSearch dan SearchResult (pengukur durasi, riwayat objective).
src/algorithms/hill_climbing.py	Implementasi varian Hill-climbing.
src/algorithms/simulated_annealing.py	Implementasi algoritma simulated annealing
src/algorithms/genetic.py	Implementasi algoritma genetic
src/experiment.py	Skenario eksperimen (menjalankan algoritma beberapa kali).
src/main.py	Entry point CLI.
src/visualization/plots.py	Visualisasi grafik objective terhadap iterasi.

2.2 Representasi State

Sebuah state direpresentasikan oleh kelas State yang menyimpan objek Truck, kamus seluruh Package (bersifat tetap dan dibagi antar-state), serta kamus placements yang memetakan id paket ke sebuah Placement. Hanya bagian placement yang disalin ketika state baru dibentuk, sehingga pembuatan neighbor cukup murah.

- position: koordinat bilangan bulat (x, y, z) sudut kiri-bawah-belakang paket, atau None jika paket berada seluruhnya di luar truk.
- orientation: permutasi dari (0, 1, 2) yang menyatakan dimensi asli mana yang sejajar dengan sumbu x, y, dan z. Terdapat 3! = 6 orientasi. Misalnya orientasi (1, 0, 2) berarti length paket sejajar sumbu x dan width sejajar sumbu y, yaitu hasil rotasi 90° terhadap sumbu z.

Rotasi 90° terhadap sebuah sumbu ekuivalen dengan menukar dua dimensi yang tegak lurus terhadap sumbu tersebut:

Sumbu rotasi	Komponen yang ditukar
x	indeks 1 dan 2
y	indeks 0 dan 2
z	indeks 0 dan 1

State awal dibangkitkan secara acak oleh State.random_initial: setiap paket diberi orientasi acak, lalu dengan peluang 0,5 diletakkan pada koordinat acak di dalam batas truk atau diletakkan di luar truk. Karena posisi diacak tanpa memperhatikan paket lain, state awal hampir selalu memiliki pelanggaran (tumpang-tindih dan melayang).

Untuk keperluan keluaran, State.to_dict menghasilkan pemetaan setiap paket ke status di dalam/di luar, posisi, orientasi, dan dimensi setelah orientasi. Hasil ini disimpan dalam berkas JSON di folder output.

2.3 Fungsi Objektif

Fungsi objektif yang digunakan didefinisikan sebagai berikut:

$$f(s) = \sum_{p \in inside(s)} skor(p) - M \cdot \sum_{k} w_k \cdot violation_k(s)$$

$$M = \sum_{p \in semua\ paket} skor(p) + 1$$

Pada model bawaan, skor(p) = value(p) sehingga pada state yang valid nilai objektif sama persis dengan total value. Nilai M lebih besar daripada total skor seluruh paket, sehingga satu pelanggaran saja duah membuat objektif lebih buruk daripada state valid mana pun, termasuk truk kosong. Akibatnya nilai objektif negatif selalu menandakan state yang tidak valid. Berikut adalah daftar jenis pelanggaran yang ada dan bagaimana cara menghitungnya.

Jenis pelanggaran	Cara penghitungan
out_of_bounds	Jumlah paket di dalam truk yang sebagian ruangnya keluar dari batas peti kemas.
overlap	Jumlah pasangan paket yang irisan ruangnya tidak kosong (uji interval terbuka pada ketiga sumbu).
floating	Jumlah paket dengan z > 0 yang tidak memiliki supporter. Paket j menjadi supporter paket i jika $z_j + h_j = z_i$ dan alas keduanya beririsan pada bidang xy.
fragile_support	Jumlah pasangan (paket, supporter) dengan supporter berupa paket pecah-belah.
overweight	Kelebihan berat total terhadap kapasitas (dalam satuan unit berat).

2.4 Fungsi Neighbor

Tiga jenis move diimplementasikan sesuai spesifikasi:

1. Swap(a, b): menukar posisi dua paket; orientasi masing-masing tetap. Karena posisi dapat bernilai None, swap antara paket di dalam dan di luar truk sekaligus memasukkan satu paket dan mengeluarkan paket lainnya.
2. Move(a, pos): memindahkan paket ke koordinat lain di dalam truk, atau mengeluarkannya dari truk (pos = None).
3. Rotate(a, axis): merotasi paket 90° terhadap sumbu x, y, atau z.

Jika move diperbolehkan ke seluruh koordinat, jumlah neighbor menjadi sangat besar (W·L·H kandidat per paket) dan sebagian besar kandidat menghasilkan paket melayang. Oleh karena itu, kandidat koordinat tujuan dibatasi pada titik (0, 0, 0) dan titik-titik sudut (extreme point) dari paket lain yang sudah berada di dalam truk: sisi kanan (x + w), sisi depan (y + l), sisi atas (z + h), dan kombinasinya. Kandidat yang membuat paket keluar dari batas truk dibuang. Heuristik ini diadaptasi dari extreme point-based heuristics untuk 3D

bin packing dan masih berada dalam lingkup local search karena hanya membatasi himpunan neighbor.

Penyaringan move yang tidak mengubah state. Fungsi is_noop membuang move yang tidak menghasilkan state baru, yaitu: swap dua paket dengan posisi sama (termasuk dua paket yang sama-sama di luar truk), move ke posisi yang sama dengan posisi saat ini, rotasi paket di luar truk, dan rotasi yang tidak mengubah dimensi (misalnya rotasi terhadap sumbu z pada paket dengan w = l).

Terdapat dua cara pembangkitan neighbor. all_moves mengenumerasi seluruh neighbor untuk Steepest Ascent, Sideways Move, dan Random Restart. random_move memilih jenis move secara acak (swap, move, atau rotate dengan peluang sama) lalu memilih parameternya secara acak, dengan percobaan ulang hingga ditemukan move yang mengubah state. Pada move untuk paket yang berada di dalam truk, peluang memilih "keluarkan dari truk" ditetapkan 50%. Tanpa aturan ini, pilihan tersebut hanya 1 dari ratusan kandidat posisi (sekitar 0,2% pada 100 paket), padahal mengeluarkan paket adalah langkah utama untuk menghilangkan tumpang-tindih pada state awal acak. Sebelum aturan ini diterapkan, Simulated Annealing pada data 100 paket tidak pernah mencapai state valid.
diturun

2.5 Evaluasi Neighbor Inkremental

Menghitung objektif dari nol berbiaya $O(n^2)$ karena harus memeriksa tumpang-tindih dan support antar semua pasangan paket. Padahal satu move hanya mengubah satu paket (move dan rotate) atau dua paket (swap). Kelas NeighborEvaluator menyimpan ringkasan state saat ini, box setiap paket, jumlah supporter setiap paket, jumlah setiap jenis pelanggaran, total value, dan total berat. Untuk setiap move, hanya pasangan yang melibatkan paket yang berubah yang dihitung ulang, termasuk perubahan status melayang paket lain yang supporter-nya berubah. Biaya evaluasi satu neighbor turun menjadi $O(n)$.

2.6 Implementasi Algoritma Local Search

Semua algoritma diturunkan dari kelas abstrak LocalSearch. Method run mengukur durasi pencarian menggunakan time.perf_counter dan membungkus hasil _search ke dalam SearchResult, yang berisi state awal dan akhir, objective awal dan akhir, banyak iterasi, riwayat objective per iterasi, serta data khusus setiap algoritma. Setiap algoritma menerima seed untuk generator bilangan acaknya sehingga eksperimen dapat direproduksi.

2.6.1 Steepest Ascent Hill-climbing

Pada setiap iterasi, fungsi best_neighbor mengevaluasi seluruh neighbor dari state saat ini dengan NeighborEvaluator lalu memilih neighbor terbaik. Jika neighbor terbaik tidak lebih besar (lebih kecil atau sama dengan) daripada nilai state saat ini, pencarian berhenti karena telah mencapai local optimum. Banyak iterasi adalah banyak perpindahan state yang dilakukan.

2.6.2 Hill-climbing with Sideways Move

Sama seperti Steepest Ascent, tetapi memiliki opsi berpindah ketika neighbor terbaik bernilai sama dengan state saat ini (sideways move), dengan harapan dapat keluar dari lokal optimum. Pencarian ini berhenti jika neighbor terbaik lebih buruk atau jika jumlah sideways move telah mencapai max_sideways

2.6.3 Random Restart Hill-climbing

Algoritma menjalankan steepest ascent dari state awal yang diberikan, lalu mengulanginya sebanyak max_restart kali, masing-masing dari state awal di generate secara acak oleh State.random_initial. State terbaik dari seluruh percobaan dikembalikan. Data tambahan yang disimpan adalah banyak restart, banyak iterasi di setiap percobaan, indeks iterasi dimulainya setiap restart dan riwayat objective terbaik saat ini.

2.6.4 Stochastic Hill-climbing

Pada setiap iterasi, satu neighbor dipilih secara acak dengan random_move. Neighbor diterima hanya jika objektifnya lebih besar daripada state saat ini. Pencarian berhenti setelah max_iterations iterasi. Karena hanya satu neighbor yang dievaluasi per iterasi, satu iterasi jauh lebih murah daripada Steepest Ascent. Banyak move yang diterma dicatat sebagai data tambahan.

2.6.5 Simulated Annealing

Berbeda dengan hill-climbing, Simulated Annealing (SA) boleh menerima neighbor yang lebih buruk dengan peluang tertentu agar dapat keluar dari local maximum. Peluang tersebut mengecil seiring turunnya temperatur T. Karena persoalan ini adalah maksimisasi, selisih energi didefinisikan sebagai

$$\Delta E = f(s') - f(s)$$

dengan s adalah state saat ini (current) dan s' adalah neighbor yang dibangkitkan. Neighbor dengan $\Delta E > 0$ selalu diterima, sedangkan neighbor dengan $\Delta E \leq 0$ diterima dengan peluang.

$$P(\text{terima}) = \begin{cases} 1, & \Delta E > 0 \\ e^{\Delta E/T}, & \Delta E \leq 0 \end{cases}$$

Pada program ini SA diimplementasikan oleh kelas SimulatedAnnealing. Pencarian dimulai dari initial state acak. Pada setiap iterasi t, temperatur dihitung dengan *cooling schedule* eksponensial

$$T_t = T_0 \cdot \alpha^t, \qquad \alpha = \left(\frac{T_{\min}}{T_0}\right)^{1/N_{\max}}$$

sehingga temperatur turun dari T_0 hingga mendekati Tmin tepat pada iterasi terakhir. Bentuk eksponensial ini diambil dari cooling schedule pada referensi**[1]**, dan desain ini dipilih karena α yang dipatok konstan (misalnya 0,999) membuat temperatur akhir bergantung pada Nmax, sedangkan α yang diturunkan dari T_0, Tmin, dan Nmax menjamin pencarian selalu berakhir pada Tmin.

2.6.6 Genetic Algorithm

Implementasi genetic algorithm dilakukan dengan membangkitkan populasi awal secara acak, lalu setiap generasi dibentuk melalui seleksi, crossover, dan mutasi. Satu individu adalah satu State. Kromosom adalah deretan gen yang diurutkan menjadi id paket, dengan satu gen untuk setiap paket. Setiap gen berupa Placement, yaitu posisi paket (koordinat (x, y, x) atau None jika paket berada di luar truk) beserta orientasinya. Karena semua individu memakai urutan id yang sama, gen ke-i pada setiap individu selalu merujuk ke paket yang sama, sehingga crossover menukar informasi untuk paket yang sama. Populasi awal terdiri atas state awal ditambah population_size - 1, state acak dari State.random_initial. Fitness setiap individu sama dengan fungsi objektif, sehingga dapat bernilai negatif untuk individu yang melanggar constraint. Individu tidak valid tetap dipertahankan, karena crossover antar-individu tidak valid masih dapat menghasilkan anak yang valid.

Induk dipilih dengan *roulette wheel selection* menggunakan bobot wi = fi - f_min + 1. Pergeseran ini membuat semua bobot positif tanpa mengubah urutan kualitas individu, dan individu terburuk tetap memiliki peluang terpilih yang kecil. Dua induk dikawinkan

dengan *single-point crossover:* gen setelah titik potong acak ditukar sehingga menghasilkan dua anak. Setiap anak kemudian dimutasi dengan peluang mutation_rate berupa satu langkah neighbor acak (random_move), sehingga mutasi selalu sesuai dengan langkah yang diizinkan.

Pada setiap generasi, satu individu terbaik dibawa langsung ke generasi berikutnya. Heuristik ini ditambahkan agar solusi terbaik tidak hilang akibat crossover atau mutasi. Sisa populasi diisi oleh anak hasil seleksi, crossover, dan mutasi. Pencarian berhenti setelah max_iterations generasi, dan individu terbaik yang pernah ditemukan dikembalikan sebagai state akhir.

2.7 Deskripsi Kelas dan Fungsi

Modul	Kelas / fungsi	Deskripsi
models	`Package`	Kelas immutable berisi id, dims, value, weight, is_fragile, eta. oriented_dims(orientation) mengembalikan dimensi setelah orientasi.
models	`Truck`	Kelas berisi dims dan max_capacity.
models	`rotate(orientation, axis)`	Rotasi 90° terhadap sumbu x/y/z dengan menukar dua komponen orientasi.
state	`Placement`	Posisi sudut kiri-bawah-belakang (None jika di luar truk) dan orientasi paket.
state	`State`	Truck, kamus paket, dan kamus placement. Method copy, inside_ids, outside_ids, box, random_initial, to_dict, from_dict.
neighbors	`Move`	Satu langkah: kind (swap/move/rotate), paket a, paket b, posisi tujuan, sumbu rotasi.
neighbors	`apply_move(state, move)`	Menghasilkan state baru (salinan) hasil penerapan move.
neighbors	`is_noop(state, move)`	Mendeteksi move yang tidak mengubah state.
neighbors	`candidate_positions(state, pid)`	Heuristik extreme point, kandidat koordinat tujuan move.
neighbors	`all_moves(state)`	Enumerasi seluruh neighbor (Steepest Ascent, Sideways, Random Restart).
neighbors	`random_move(state, rng)`	Memilih satu neighbor acak (Stochastic HC, SA, mutasi GA).
objective	`count_violations(state)`	Menghitung jumlah tiap jenis pelanggaran constraint.
objective	`objective(state)`	Total skor paket di dalam truk dikurangi penalti pelanggaran.
objective	`set_objective_mode(mode)`	Memilih mode objective: value (default) atau density (value/berat).

Modul	Kelas / fungsi	Deskripsi
evaluator	`NeighborEvaluator`	Menyimpan ringkasan pelanggaran state; evaluate(move) menghitung objective neighbor dalam O(n).
base	`LocalSearch`	Kelas abstrak; run() mengukur durasi dan membungkus hasil _search().
base	`SearchResult`	State awal/akhir, objective awal/akhir, durasi, iterasi, history, data tambahan, mode objective.
hill_climbing	`best_neighbor(state)`	Mengevaluasi seluruh neighbor dan mengembalikan yang objektifnya terbesar.
hill_climbing	`SteepestAscentHC`	Steepest Ascent Hill-climbing.
hill_climbing	`SidewaysMoveHC`	Hill-climbing with Sideways Move, parameter max_sideways.
hill_climbing	`RandomRestartHC`	Random Restart Hill-climbing, parameter max_restart.
hill_climbing	`StochasticHC`	Stochastic Hill-climbing, parameter max_iterations.
simulated_annealing	`SimulatedAnnealing`	Parameter initial_temp, cooling_rate (opsional. Jika kosong dihitung dari initial_temp, min_temp, dan max_iterations), min_temp, max_iterations, stuck_patience. Method schedule(t) dan acceptance_probability(ΔE, T).
genetic	`GeneticAlgorithm`	Parameter population_size, max_iterations, mutation_rate, elitism. Method init_population, fitness, select, crossover, mutate.
experiment	`run_repeated, run_ga_experiment`	Menjalankan algoritma 3 kali dengan seed berbeda, dan seluruh skenario variasi GA.
plots	`plot_state, plot_history, plot_acceptance_probability`	Visualisasi state 3D (dapat diputar), objective terhadap iterasi, dan $e^{(\Delta E/T)}$ terhadap iterasi.

Tabel 5 Deskripsi kelas dan fungsi utama

2.8 Visualisasi

Modul `visualization/plots.py` menyediakan tiga jenis visualisasi yang disimpan otomatis untuk setiap run di folder output/<n>_paket:

- `plot_state`: gambar 3D peti kemas beserta seluruh paket di dalamnya. Warna setiap paket tetap (berdasarkan urutan id) sehingga sama pada state awal dan akhir, paket pecah-belah diarsir, dan panel di sebelahnya menampilkan objective, total value, berat, jumlah paket, status valid, rincian pelanggaran, serta daftar paket di luar truk.

- `plot_history`: objective terhadap iterasi. Untuk GA ditampilkan objective maksimum dan rata-rata populasi; untuk Random Restart ditampilkan juga objective terbaik sejauh ini.
- `plot_acceptance_probability`: $e^{(\Delta E/T)}$ terhadap iterasi untuk Simulated Annealing.

Dengan opsi --show, plot state ditampilkan dalam jendela interaktif yang dapat diputar 360° dengan menyeret mouse, menggeser slider azimuth (0–360°) dan elevasi, atau menekan tombol putar otomatis. State hasil run yang sudah tersimpan dapat dibuka ulang tanpa menjalankan algoritma dengan python view_state.py --input <problem.json> --result <hasil.json>.

2.9 Source Code

Berikut source code yang telah diimplementasikan

src/core/models.py

```
from __future__ import annotations

from dataclasses import dataclass
from itertools import permutations

Dim = tuple[int, int, int]

ORIENTATIONS: list[Dim] = list(permutations((0, 1, 2)))

ROTATION_SWAP = {
    "x": (1, 2),
    "y": (0, 2),
    "z": (0, 1),
}

@dataclass(frozen=True)
class Package:
    id: str
    dims: Dim
    value: int
    weight: int
    is_fragile: bool
    eta: int

    def oriented_dims(self, orientation: Dim) -> Dim:
        return tuple(self.dims[i] for i in orientation)

    @property
    def volume(self) -> int:
        w, l, h = self.dims
        return w * l * h

@dataclass(frozen=True)
class Truck:
```

```
    dims: Dim
    max_capacity : int

    @property
    def volume(self) -> int:
        w, l, h = self.dims
        return w * l * h

def rotate(orientation: Dim, axis: str) -> Dim:
    a, b = ROTATION_SWAP[axis]
    o = list(orientation)
    o[a], o[b] = o[b], o[a]
    return tuple(o)
```

src/core/state.py

```
from __future__ import annotations

import random
from dataclasses import dataclass, field

from .models import ORIENTATIONS, Dim, Package, Truck

@dataclass
class Placement:
    position: Dim | None = None
    orientation: Dim = (0, 1, 2)

    @property
    def inside(self) -> bool:
        return self.position is not None

@dataclass
class State:
    truck: Truck
    packages: dict[str, Package]
    placements: dict[str, Placement] = field(default_factory=dict)

    def copy(self) -> "State":
        return State(
            truck= self.truck,
            packages=self.packages,
            placements={
                pid: Placement(p.position, p.orientation)
                for pid, p in self.placements.items()
            },
        )

    def inside_ids(self) -> list[str]:
        return [pid for pid, p in self.placements.items() if p.inside]

    def outside_ids(self) -> list[str]:
        return [pid for pid, p in self.placements.items() if not p.inside]

    def box(self, pid: str) -> tuple[Dim, Dim]:
        pl = self.placements[pid]
        assert pl.position is not None, f"{pid} berada di luar truk"
        return pl.position, self.packages[pid].oriented_dims(pl.orientation)

    @classmethod
```

```
    def random_initial(
        cls,
        truck: Truck,
        packages: dict[str, Package],
        rng: random.Random | None = None,
    ) -> "State":
        """TODO: Apakah perlu di pilih orientasi yang muat kalo tidak muat?"""
        rng = rng or random.Random()
        state = cls(truck=truck, packages=packages)
        truck_w, truck_d, truck_h = truck.dims

        for pid, package in packages.items():
            # pilih orientasi random
            orientation = rng.choice(ORIENTATIONS)

            # dimensi paket setelah di orientasikan
            w, d, h = package.oriented_dims(orientation)

            if w > truck_w or d > truck_d or h > truck_h:
                state.placements[pid] = Placement(position=None, orientation=orientation,)
                continue

            # paket random bisa di luar atau di dalam

            if rng.random() < 0.5:
                x = rng.randint(0, truck_w-w)
                y = rng.randint(0, truck_d-d)
                z = rng.randint(0, truck_h-h)
                position = (x, y, z)

            else:
                position = None

            state.placements[pid] = Placement(position=position, orientation=orientation)
        return state

    @classmethod
    def from_dict(cls, truck: Truck, packages: dict[str, Package], data: dict) -> "State":
        """Kebalikan to_dict: bangun ulang state dari hasil run yang disimpan."""
        state = cls(truck, packages)
        state.placements = {
            pid: Placement(position=tuple(p["position"]) if p["position"] is not None else
None,
                           orientation=tuple(p["orientation"]))
            for pid, p in data.items()
        }
        return state

    def to_dict(self) -> dict:
        return {
            pid: {
                "inside": pl.inside,
                "position": list(pl.position) if pl.position else None,
                "orientation": list(pl.orientation),
                "oriented_dims": list(self.packages[pid].oriented_dims(pl.orientation)),
            }
            for pid, pl in self.placements.items()
        }
```

src/core/neighbors.py

```
from __future__ import annotations

import random
from collections.abc import Iterator
from dataclasses import dataclass

from .models import Dim, rotate
from .state import State

@dataclass(frozen=True)
class Move:
    kind: str # "swap" | "move" | "rotate"
    a: str # id paket
    b: str | None = None # id paket kedua kalo swap
    position: Dim | None = None # target koordinat kalo move, None = keluarkan dari Truk
    axis: str | None = None # "x" | "y" | "z"

def apply_move(state: State, move: Move) -> State:
    new = state.copy()
    pa = new.placements[move.a]

    if move.kind == "swap":
        pb = new.placements[move.b]
        # tukar posisi saja, orientasi tetap
        pa.position, pb.position = pb.position, pa.position
    elif move.kind == "move":
        pa.position = move.position
    elif move.kind == "rotate":
        pa.orientation = rotate(pa.orientation, move.axis)
    else:
        raise ValueError(f"Move tidak dikenal: {move.kind}")
    return new

def is_noop(state: State, move: Move) -> bool:
    """True jika move tidak mengubah state (tidak dihitung sebagai neighbor)."""
    pa = state.placements[move.a]

    if move.kind == "swap":
        # posisi sama (termasuk sama-sama di luar truk) -> tidak ada perubahan
        return pa.position == state.placements[move.b].position
    if move.kind == "move":
        return pa.position == move.position
    if move.kind == "rotate":
        # rotasi paket di luar truk tidak berpengaruh, begitu juga rotasi
        # yang menghasilkan dimensi sama (contoh sisi w == l saat rotasi z)
        if not pa.inside:
            return True
        package = state.packages[move.a]
        return (package.oriented_dims(rotate(pa.orientation, move.axis))
                == package.oriented_dims(pa.orientation))
    raise ValueError(f"Move tidak dikenal: {move.kind}")

def candidate_positions(state: State, pid: str) -> list[Dim]:
    truck_w, truck_d, truck_h = state.truck.dims

    package = state.packages[pid]
    orientation = state.placements[pid].orientation
    w, d, h = package.oriented_dims(orientation)
```

```
    candidates: set[Dim] = set()

    # Kalau paket lebih besar dari truck dalam orientasi ini,
    # tidak ada posisi valid di dalam truck.
    if w > truck_w or d > truck_d or h > truck_h:
        return []

    # Pojok/lantai truck
    candidates.add((0, 0, 0))

    # Extreme points berdasarkan paket lain
    for other in state.inside_ids():
        if other == pid:
            continue

        (x, y, z), (ow, od, oh) = state.box(other)

        # Di sebelah kanan paket
        candidates.add((x + ow, y, z))

        # Di depan paket
        candidates.add((x, y + od, z))

        # Di atas paket
        candidates.add((x, y, z + oh))

        # Kombinasi pojok
        candidates.add((x + ow, y + od, z))
        candidates.add((x + ow, y, z + oh))
        candidates.add((x, y + od, z + oh))
        candidates.add((x + ow, y + od, z + oh))

    # Batasi hanya posisi yang seluruh box-nya berada di dalam truck
    valid_candidates = []

    for x, y, z in candidates:
        if (
            0 <= x
            and x + w <= truck_w
            and 0 <= y
            and y + d <= truck_d
            and 0 <= z
            and z + h <= truck_h
        ):
            valid_candidates.append((x, y, z))

    return valid_candidates

def all_moves(state: State) -> Iterator[Move]:
    """Enumerasi seluruh neighbor (dipakai steepest ascent / sideways / random
restart)."""
    for move in _all_moves_raw(state):
        if not is_noop(state, move):
            yield move

def _all_moves_raw(state: State) -> Iterator[Move]:
    ids = list(state.placements)
    # swap
    for i, a in enumerate(ids):
        for b in ids[i + 1:]:
```

```
            yield Move("swap", a, b=b)
    # rotate
    for a in ids:
        for axis in ("x", "y", "z"):
            yield Move("rotate", a, axis=axis)
    # move
    for a in ids:
        for pos in candidate_positions(state, a):
            yield Move("move", a, position=pos)
        if state.placements[a].inside:
            yield Move("move", a, position=None)  # keluarkan dari truk

def random_move(state: State, rng: random.Random, max_tries: int = 100) -> Move:
    """Ambil satu neighbor secara acak (move yang tidak mengubah state dilewati)."""
    ids = list(state.placements)

    if not ids:
        raise ValueError("State tidak memiliki paket.")

    for _ in range(max_tries):
        kind = rng.choice(("swap", "move", "rotate"))

        # swap
        if kind == "swap":
            if len(ids) < 2:
                continue
            a, b = rng.sample(ids, 2)
            move = Move(kind="swap", a=a, b=b)

        # move
        elif kind == "move":
            a = rng.choice(ids)
            candidates = candidate_positions(state, a)

            if state.placements[a].inside and (not candidates or rng.random() < 0.5):
                move = Move(kind="move", a=a, position=None)
            elif candidates:
                move = Move(kind="move", a=a, position=rng.choice(candidates))
            else:
                continue

        # rotate (hanya berpengaruh untuk paket di dalam truk)
        else:
            inside = state.inside_ids()
            if not inside:
                continue
            move = Move(kind="rotate", a=rng.choice(inside),
                        axis=rng.choice(("x", "y", "z")))

        if not is_noop(state, move):
            return move

    raise ValueError("Tidak ditemukan neighbor yang mengubah state.")
```

src/core/objective.py

```
from __future__ import annotations

from .models import Package
from .state import State
```

```
PENALTY_WEIGHTS = {
    "out_of_bounds": 1,
    "overlap": 1,
    "floating": 1,
    "fragile_support": 1,
    "overweight": 1,
}

OBJECTIVE_MODES = ("value", "density")
_objective_mode = "value"

_ROUND_DIGITS = 6


def set_objective_mode(mode: str) -> None:
    global _objective_mode
    if mode not in OBJECTIVE_MODES:
        raise ValueError(f"Mode objective tidak dikenal: {mode}")
    _objective_mode = mode


def get_objective_mode() -> str:
    return _objective_mode


def package_score(package: Package) -> float:
    if _objective_mode == "density":
        return package.value / package.weight if package.weight > 0 else package.value
    return package.value


# agar value ketika ada pelanggaran selalu lebih buruk dibandingkan tidak ada
def penalty_scale(state: State) -> float:
    return sum(package_score(p) for p in state.packages.values()) + 1

def is_within_truck(state: State, pid: str) -> bool:
    pl = state.placements[pid]

    if pl.position is None:
        return False

    truck_w, truck_d, truck_h = state.truck.dims
    (x, y, z), (w, d, h) = state.box(pid)
    return (x >= 0 and x + w <= truck_w and y >= 0 and y + d <= truck_d
            and z >= 0 and z + h <= truck_h)

def overlaps(state: State, a: str, b: str) -> bool:
    if not state.placements[a].inside or not state.placements[b].inside:
        return False

    (x1, y1, z1), (w1, d1, h1) = state.box(a)
    (x2, y2, z2), (w2, d2, h2) = state.box(b)

    return not (x1 + w1 <= x2 or x2 + w2 <= x1
            or y1 + d1 <= y2 or y2 + d2 <= y1
            or z1 + h1 <= z2 or z2 + h2 <= z1)

def supporters(state: State, pid: str) -> list[str]:
    if not state.placements[pid].inside:
```

```
        return []

    (xi, yi, zi), (wi, di, hi) = state.box(pid)

    result = []

    for other in state.inside_ids():
        if other == pid:
            continue

        (xj, yj, zj), (wj, dj, hj) = state.box(other)

        if zj + hj != zi:
            continue

        x_overlap = (
            xi < xj + wj
            and xj < xi + wi
        )

        y_overlap = (
            yi < yj + dj
            and yj < yi + di
        )

        if x_overlap and y_overlap:
            result.append(other)

    return result

def is_floating(state: State, pid: str) -> bool:
    if not state.placements[pid].inside:
        return False

    _, _, zi = state.placements[pid].position

    if zi == 0:
        return False

    return len(supporters(state, pid)) == 0

def total_weight(state: State) -> int:
    return sum(state.packages[pid].weight for pid in state.inside_ids())


def total_value(state: State) -> int:
    return sum(state.packages[pid].value for pid in state.inside_ids())


def total_score(state: State) -> float:
    return sum(package_score(state.packages[pid]) for pid in state.inside_ids())


def count_violations(state: State) -> dict[str, int]:
    inside = state.inside_ids()

    # paket di dalam yang menembus truk
    out_of_bounds = sum(not is_within_truck(state, pid) for pid in inside)

    # hitung yg overlap
```

```
    overlap = 0

    for i in range(len(inside)):
        for j in range(i+1, len(inside)):
            if overlaps(state, inside[i], inside[j]):
                overlap +=1

    # supporters tiap paket dihitung sekali, dipakai untuk cek melayang dan fragile
    sup = {pid: supporters(state, pid) for pid in inside}

    # hitung melayang
    floating = sum(
        state.placements[pid].position[2] != 0 and not sup[pid]
        for pid in inside
    )

    # paket fragile yang jadi support
    fragile_support = sum(
        state.packages[s].is_fragile
        for pid in inside
        for s in sup[pid]
    )

    # berat melebihi kapasitas
    overweight = max(0, total_weight(state) - state.truck.max_capacity)

    return {
        "out_of_bounds": out_of_bounds,
        "overlap": overlap,
        "floating": floating,
        "fragile_support": fragile_support,
        "overweight": overweight,
    }

def is_valid(state: State) -> bool:
    return all(v == 0 for v in count_violations(state).values())

def objective(state: State) -> float:
    violations = count_violations(state)
    scale = penalty_scale(state)
    penalty = sum(scale * PENALTY_WEIGHTS[k] * v for k, v in violations.items())
    return round(total_score(state) - penalty, _ROUND_DIGITS)
```

src/core/evaluator.py

```
from __future__ import annotations

from .models import Dim, rotate
from .neighbors import Move
from .objective import (_ROUND_DIGITS, PENALTY_WEIGHTS, count_violations, package_score,
                        penalty_scale, supporters, total_score)
from .state import State

Box = tuple[Dim, Dim]


def _out_of_bounds(box: Box, truck_dims: Dim) -> bool:
    (x, y, z), (w, d, h) = box
    tw, td, th = truck_dims
    return not (x >= 0 and x + w <= tw and y >= 0 and y + d <= td and z >= 0 and z + h
```

```
<= th)


def _overlap(a: Box, b: Box) -> bool:
    (x1, y1, z1), (w1, d1, h1) = a
    (x2, y2, z2), (w2, d2, h2) = b
    return not (x1 + w1 <= x2 or x2 + w2 <= x1
                or y1 + d1 <= y2 or y2 + d2 <= y1
                or z1 + h1 <= z2 or z2 + h2 <= z1)


def _supports(lower: Box, upper: Box) -> bool:
    (xl, yl, zl), (wl, dl, hl) = lower
    (xu, yu, zu), (wu, du, _) = upper
    return (zl + hl == zu
            and xu < xl + wl and xl < xu + wu
            and yu < yl + dl and yl < yu + du)


class NeighborEvaluator:
    def __init__(self, state: State):
        self.state = state
        self.scale = penalty_scale(state)
        self.inside = state.inside_ids()
        self.boxes: dict[str, Box] = {pid: state.box(pid) for pid in self.inside}
        self.sup_count = {pid: len(supporters(state, pid)) for pid in self.inside}
        self.violations = count_violations(state)
        self.value = total_score(state)
        self.weight = sum(state.packages[pid].weight for pid in self.inside)

    def _new_boxes(self, move: Move) -> dict[str, Box | None]:
        pls, pkgs = self.state.placements, self.state.packages
        pa = pls[move.a]

        def box(pid: str, pos: Dim | None, orientation: Dim) -> Box | None:
            return None if pos is None else (pos, pkgs[pid].oriented_dims(orientation))

        if move.kind == "swap":
            pb = pls[move.b]
            return {move.a: box(move.a, pb.position, pa.orientation),
                    move.b: box(move.b, pa.position, pb.orientation)}
        if move.kind == "move":
            return {move.a: box(move.a, move.position, pa.orientation)}
        if move.kind == "rotate":
            return {move.a: box(move.a, pa.position, rotate(pa.orientation, move.axis))}
        raise ValueError(f"Move tidak dikenal: {move.kind}")

    def evaluate(self, move: Move) -> float:
        pkgs = self.state.packages
        truck = self.state.truck
        new = self._new_boxes(move)
        old = {pid: self.boxes.get(pid) for pid in new}
        changed = list(new)

        d = dict.fromkeys(self.violations, 0)
        d_value = d_weight = 0
        sup_delta: dict[str, int] = {}          # perubahan jumlah supporter paket lain
        new_sup = dict.fromkeys(changed, 0)     # jumlah supporter baru paket yang
berubah
```

```
        for c in changed:
            ob, nb = old[c], new[c]
            pkg = pkgs[c]
            frag_c = pkg.is_fragile
            if ob is not None:
                d_value -= package_score(pkg); d_weight -= pkg.weight
                d["out_of_bounds"] -= _out_of_bounds(ob, truck.dims)
            if nb is not None:
                d_value += package_score(pkg); d_weight += pkg.weight
                d["out_of_bounds"] += _out_of_bounds(nb, truck.dims)

            # pasangan dengan paket yang tidak berubah
            for j in self.inside:
                if j in new:
                    continue
                jb = self.boxes[j]
                frag_j = pkgs[j].is_fragile
                if ob is not None:
                    d["overlap"] -= _overlap(ob, jb)
                    if _supports(ob, jb):
                        d["fragile_support"] -= frag_c
                        sup_delta[j] = sup_delta.get(j, 0) - 1
                    if _supports(jb, ob):
                        d["fragile_support"] -= frag_j
                if nb is not None:
                    d["overlap"] += _overlap(nb, jb)
                    if _supports(nb, jb):
                        d["fragile_support"] += frag_c
                        sup_delta[j] = sup_delta.get(j, 0) + 1
                    if _supports(jb, nb):
                        d["fragile_support"] += frag_j
                        new_sup[c] += 1

        # pasangan antar paket yang berubah (swap)
        if len(changed) == 2:
            a, b = changed
            for boxes, sign in ((old, -1), (new, 1)):
                ba, bb = boxes[a], boxes[b]
                if ba is None or bb is None:
                    continue
                d["overlap"] += sign * _overlap(ba, bb)
                if _supports(ba, bb):
                    d["fragile_support"] += sign * pkgs[a].is_fragile
                    if sign > 0:
                        new_sup[b] += 1
                if _supports(bb, ba):
                    d["fragile_support"] += sign * pkgs[b].is_fragile
                    if sign > 0:
                        new_sup[a] += 1

        # melayang: paket yang berubah, dan paket lain yang supporter-nya berubah
        for c in changed:
            ob, nb = old[c], new[c]
            if ob is not None:
                d["floating"] -= ob[0][2] != 0 and self.sup_count[c] == 0
            if nb is not None:
                d["floating"] += nb[0][2] != 0 and new_sup[c] == 0
        for j, delta in sup_delta.items():
            if delta and self.boxes[j][0][2] != 0:
                before = self.sup_count[j]
```

```
                d["floating"] += (before + delta == 0) - (before == 0)

        weight = self.weight + d_weight
        d["overweight"] = (max(0, weight - truck.max_capacity)
                           - self.violations["overweight"])

        penalty = sum(self.scale * PENALTY_WEIGHTS[k] * (self.violations[k] + d[k])
                      for k in self.violations)
        return round(self.value + d_value - penalty, _ROUND_DIGITS)
```

src/algoritmhs/base.py

```
from __future__ import annotations

import random
import time
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import Any

from core.objective import get_objective_mode, objective, total_value
from core.state import State


@dataclass
class SearchResult:
    algorithm: str
    initial_state: State
    final_state: State
    initial_objective: float
    final_objective: float
    duration_s: float
    iterations: int
    history: list[float] = field(default_factory=list)   # objective per iterasi
    extra: dict[str, Any] = field(default_factory=dict)  # data khusus per algoritma
    objective_mode: str = "value"

    def to_dict(self) -> dict:
        return {
            "algorithm": self.algorithm,
            "objective_mode": self.objective_mode,
            "final_total_value": total_value(self.final_state),
            "initial_objective": self.initial_objective,
            "final_objective": self.final_objective,
            "duration_s": self.duration_s,
            "iterations": self.iterations,
            "history": self.history,
            "extra": self.extra,
            "initial_state": self.initial_state.to_dict(),
            "final_state": self.final_state.to_dict(),
        }


class LocalSearch(ABC):
    name = "local_search"

    def __init__(self, seed: int | None = None):
        self.rng = random.Random(seed)
```

```
    def run(self, initial: State) -> SearchResult:
        """Wrapper yang mengukur durasi; logika ada di `_search`."""
        start = time.perf_counter()
        final, iterations, history, extra = self._search(initial)
        duration = time.perf_counter() - start
        return SearchResult(
            algorithm=self.name,
            initial_state=initial,
            final_state=final,
            initial_objective=objective(initial),
            final_objective=objective(final),
            duration_s=duration,
            iterations=iterations,
            history=history,
            extra=extra,
            objective_mode=get_objective_mode(),
        )

    @abstractmethod
    def _search(self, initial: State) -> tuple[State, int, list[float], dict[str, Any]]:
        """Kembalikan (final_state, banyak_iterasi, history_objective, extra)."""
```

src/algoritmhs/hill_climbing.py

```
from __future__ import annotations
from typing import Any

from core.evaluator import NeighborEvaluator
from core.neighbors import all_moves, apply_move, random_move
from core.objective import objective
from core.state import State
from .base import LocalSearch

def best_neighbor(state: State) -> tuple[State | None, float]:
    evaluator = NeighborEvaluator(state)
    best_move, best_val = None, float("-inf")
    for move in all_moves(state):
        val = evaluator.evaluate(move)
        if val > best_val:
            best_move, best_val = move, val
    if best_move is None:
        return None, best_val
    return apply_move(state, best_move), best_val

class SteepestAscentHC(LocalSearch):
    name = "steepest_ascent_hc"

    def _search(self, initial: State) -> tuple[State, int, list[float], dict[str, Any]]:
        current, current_val = initial, objective(initial)
        history = [current_val]
        iterations = 0
        while True:
            neighbor, val = best_neighbor(current)
            if val <= current_val:
                break

            current = neighbor
            current_val = val
```

```
            iterations += 1
            history.append(current_val)

        return current, iterations, history, {}


class SidewaysMoveHC(LocalSearch):
    name = "sideways_move_hc"

    def __init__(self, max_sideways: int = 100, seed: int | None = None):
        super().__init__(seed)
        self.max_sideways = max_sideways

    def _search(self, initial):
        current, current_val = initial, objective(initial)
        history = [current_val]
        iterations = 0
        sideways_count = 0
        while True:
            neighbor, val = best_neighbor(current)
            if val < current_val:
                break

            if val == current_val:
                if sideways_count >= self.max_sideways:
                    break
                sideways_count+=1

            current = neighbor
            current_val = val

            iterations += 1
            history.append(current_val)

        return current, iterations, history, {}

class RandomRestartHC(LocalSearch):
    name = "random_restart_hc"

    def __init__(self, max_restart: int = 10, seed: int | None = None):
        super().__init__(seed)
        self.max_restart = max_restart

    def _search(self, initial: State) -> tuple[State, int, list[float], dict[str, Any]]:
        best, best_val = initial, objective(initial)
        history = []
        best_history = []
        iterations_per_restart = []
        restart_at = []  # indeks history tempat restart dimulai (untuk plot)

        for attempt in range(self.max_restart + 1):
            if attempt == 0:
                current = initial
            else:
                current = State.random_initial(initial.truck, initial.packages,
self.rng)
                restart_at.append(len(history))
            current_val = objective(current)
            history.append(current_val)
            best_history.append(max(best_val, current_val))
```

```
            iterations = 0

            while True:
                neighbor, val = best_neighbor(current)
                if val <= current_val:
                    break

                current = neighbor
                current_val = val

                iterations += 1
                history.append(current_val)
                best_history.append(max(best_history[-1], current_val))

            iterations_per_restart.append(iterations)
            if current_val > best_val:
                best, best_val = current, current_val

        extra = {
            "restarts": self.max_restart,
            "iterations_per_restart": iterations_per_restart,
            "restart_at": restart_at,
            "best_history": best_history,
        }
        return best, sum(iterations_per_restart), history, extra


class StochasticHC(LocalSearch):
    name = "stochastic_hc"

    def __init__(self, max_iterations: int = 10_000, seed: int | None = None):
        super().__init__(seed)
        self.max_iterations = max_iterations

    def _search(self, initial: State) -> tuple[State, int, list[float], dict[str, Any]]:
        current, current_val = initial , objective(initial)
        evaluator = NeighborEvaluator(current)
        history = [current_val]
        accepted = 0
        iterations = 0

        for _ in range(self.max_iterations):
            try:
                move = random_move(current, self.rng)
            except ValueError:
                # tidak ada neighbor yang mengubah state
                break
            iterations += 1
            val = evaluator.evaluate(move)
            if val > current_val:
                current, current_val = apply_move(current, move), val
                evaluator = NeighborEvaluator(current)
                accepted += 1
            history.append(current_val)

        return current, iterations, history, {"accepted_moves": accepted}
```

src/algorithms/simulated_annealing.py

```
from __future__ import annotations
from typing import Any

from core.evaluator import NeighborEvaluator
from core.neighbors import apply_move, random_move
from core.objective import objective
from core.state import State
from .base import LocalSearch

import math

class SimulatedAnnealing(LocalSearch):
    name = "simulated_annealing"

    def __init__(
        self,
        initial_temp: float = 1000.0,
        cooling_rate: float | None = None,
        min_temp: float = 0.001,
        max_iterations: int = 5000,
        stuck_patience: int = 100,
        seed: int | None = None,
    ):
        super().__init__(seed)
        self.initial_temp = initial_temp
        self.min_temp = min_temp
        self.max_iterations = max_iterations
        self.cooling_rate = (cooling_rate if cooling_rate is not None
                             else (min_temp / initial_temp) ** (1 / max_iterations))
        self.stuck_patience = stuck_patience

    def schedule(self, t: int) -> float:
        return self.initial_temp * (self.cooling_rate ** t)

    @staticmethod
    def acceptance_probability(delta_e: float, temp: float) -> float:
        if delta_e > 0:
            return 1.0
        return math.exp(delta_e / temp) if temp > 0 else 0.0

    def _search(self, initial: State) -> tuple[State, int, list[float], dict[str, Any]]:
        current, current_val = initial, objective(initial)
        evaluator = NeighborEvaluator(current)
        best, best_val = current, current_val
        history = [current_val]
        best_history = [best_val]
        prob_history: list[float] = []
        stuck_events: list[tuple[int, float]] = []
        stuck_iterations = 0
        rejected_valid = 0
        rejected_invalid = 0
        unchanged_streak = 0
        iterations = 0

        for t in range(self.max_iterations):
            temp = self.schedule(t)
            if temp < self.min_temp:
                break

            try:
```

```
                move = random_move(current, self.rng)
            except ValueError:
                break

            neighbor_val = evaluator.evaluate(move)
            delta_e = neighbor_val - current_val
            p = self.acceptance_probability(delta_e, temp)
            prob_history.append(p)
            iterations += 1

            previous_val = current_val
            if self.rng.random() < p:
                current, current_val = apply_move(current, move), neighbor_val
                evaluator = NeighborEvaluator(current)
                if current_val > best_val:
                    best, best_val = current, current_val
            elif neighbor_val < 0:
                rejected_invalid += 1
            else:
                rejected_valid += 1

            if current_val == previous_val:
                unchanged_streak += 1
                if unchanged_streak == self.stuck_patience:
                    stuck_events.append((iterations, temp))
                if unchanged_streak >= self.stuck_patience:
                    stuck_iterations += 1
            else:
                unchanged_streak = 0

            history.append(current_val)
            best_history.append(best_val)

        return best, iterations, history, {
            "acceptance_probability": prob_history,
            "best_history": best_history,
            "stuck_count": len(stuck_events),
            "stuck_events": stuck_events,
            "stuck_iterations": stuck_iterations,
            "stuck_patience": self.stuck_patience,
            "rejected_valid": rejected_valid,
            "rejected_invalid": rejected_invalid,
        }
```

src/algorithms/genetic.py

```
"""Genetic Algorithm.

Parameter yang divariasikan: jumlah populasi dan banyak iterasi (generasi).
"""

from __future__ import annotations

from typing import Any

from core.neighbors import apply_move, random_move
from core.objective import objective
from core.state import Placement, State
```

```
from .base import LocalSearch


class GeneticAlgorithm(LocalSearch):
    name = "genetic_algorithm"

    def __init__(
        self,
        population_size: int = 50,
        max_iterations: int = 100,
        mutation_rate: float = 0.1,
        elitism: int = 1,
        seed: int | None = None,
    ):
        super().__init__(seed)
        if population_size < 2:
            raise ValueError("population_size minimal 2 agar crossover bisa dilakukan.")
        self.population_size = population_size
        self.max_iterations = max_iterations
        self.mutation_rate = mutation_rate
        self.elitism = min(elitism, population_size)

    def init_population(self, initial: State) -> list[State]:
        """Populasi awal: `initial` + State.random_initial(...) sebanyak population_size
- 1."""
        population = [initial]
        for _ in range(self.population_size - 1):
            population.append(State.random_initial(initial.truck, initial.packages,
self.rng))
        return population

    def fitness(self, state: State) -> float:
        return objective(state)

    def select(self, population: list[State], fitnesses: list[float]) -> State:
        """Roulette wheel: peluang terpilih sebanding dengan fitness.
        Fitness digeser (f - f_min + 1) agar semua bobot positif."""
        f_min = min(fitnesses)
        weights = [f - f_min + 1 for f in fitnesses]
        return self.rng.choices(population, weights=weights, k=1)[0]

    def crossover(self, a: State, b: State) -> tuple[State, State]:
        """Single-point crossover: gen (placement per paket) setelah titik potong
ditukar."""
        ids = list(a.placements)
        if len(ids) < 2:
            return a.copy(), b.copy()

        point = self.rng.randint(1, len(ids) - 1)
        c1, c2 = a.copy(), b.copy()
        for pid in ids[point:]:
            pa, pb = a.placements[pid], b.placements[pid]
            c1.placements[pid] = Placement(pb.position, pb.orientation)
            c2.placements[pid] = Placement(pa.position, pa.orientation)
        return c1, c2

    def mutate(self, state: State) -> State:
        """Dengan peluang mutation_rate, terapkan satu langkah neighbor acak
(swap/move/rotate)."""
        if self.rng.random() >= self.mutation_rate:
```

```
            return state
        try:
            return apply_move(state, random_move(state, self.rng))
        except ValueError:
            # tidak ada neighbor yang mengubah state
            return state

    def _search(self, initial: State) -> tuple[State, int, list[float], dict[str, Any]]:
        max_history: list[float] = []  # objective maksimum per generasi
        avg_history: list[float] = []  # objective rata-rata per generasi

        population = self.init_population(initial)
        fitnesses = [self.fitness(s) for s in population]

        best_idx = max(range(len(population)), key=fitnesses.__getitem__)
        best, best_val = population[best_idx], fitnesses[best_idx]
        initial_best_val = best_val

        max_history.append(max(fitnesses))
        avg_history.append(sum(fitnesses) / len(fitnesses))

        for _ in range(self.max_iterations):
            # elitism: individu terbaik langsung dibawa ke generasi berikutnya
            ranked = sorted(range(len(population)), key=fitnesses.__getitem__,
reverse=True)
            new_pop = [population[i] for i in ranked[:self.elitism]]

            while len(new_pop) < self.population_size:
                p1 = self.select(population, fitnesses)
                p2 = self.select(population, fitnesses)
                c1, c2 = self.crossover(p1, p2)
                new_pop += [self.mutate(c1), self.mutate(c2)]

            population = new_pop[:self.population_size]
            fitnesses = [self.fitness(s) for s in population]

            gen_best = max(range(len(population)), key=fitnesses.__getitem__)
            if fitnesses[gen_best] > best_val:
                best, best_val = population[gen_best], fitnesses[gen_best]

            max_history.append(max(fitnesses))
            avg_history.append(sum(fitnesses) / len(fitnesses))

        return best, self.max_iterations, max_history, {
            "population_size": self.population_size,
            "mutation_rate": self.mutation_rate,
            "elitism": self.elitism,
            "initial_best_objective": initial_best_val,
            "avg_history": avg_history,
        }
```

src/experiments.py

```
from __future__ import annotations

import random
from pathlib import Path

from algorithms import ALGORITHMS
```

```
from algorithms.base import SearchResult
from algorithms.genetic import GeneticAlgorithm
from core.io_utils import save_result
from core.models import Package, Truck
from core.objective import is_valid, total_value
from core.state import State

N_RUNS = 3

GA_CONTROL_POPULATION = 50
GA_ITERATION_VARIANTS = (50, 100, 200)
GA_CONTROL_ITERATIONS = 100
GA_POPULATION_VARIANTS = (20, 50, 100)

def _seed_for(seed: int | None, i: int) -> int | None:
    return None if seed is None else seed + i

def run_repeated(algo_key: str, truck: Truck, packages: dict[str, Package],
                 n_runs: int = N_RUNS, seed: int | None = None,
                 **params) -> list[tuple[SearchResult, Path]]:
    results = []
    for i in range(n_runs):
        run_seed = _seed_for(seed, i)
        initial = State.random_initial(truck, packages, random.Random(run_seed))
        algo = ALGORITHMS[algo_key](seed=run_seed, **params)
        result = algo.run(initial)
        path = save_result(result.to_dict(), f"{algo.name}_run{i + 1}")
        results.append((result, path))
    return results

def run_ga_experiment(truck: Truck, packages: dict[str, Package],
                      n_runs: int = N_RUNS,
                      seed: int | None = None) -> list[tuple[SearchResult, Path]]:
    configs = [(GA_CONTROL_POPULATION, it) for it in GA_ITERATION_VARIANTS]
    configs += [(pop, GA_CONTROL_ITERATIONS) for pop in GA_POPULATION_VARIANTS]
    configs = list(dict.fromkeys(configs))

    results = []
    for pop, it in configs:
        for i in range(n_runs):
            run_seed = _seed_for(seed, i)
            initial = State.random_initial(truck, packages, random.Random(run_seed))
            algo = GeneticAlgorithm(population_size=pop, max_iterations=it,
seed=run_seed)
            result = algo.run(initial)
            path = save_result(result.to_dict(), f"{algo.name}_pop{pop}_iter{it}_run{i +
1}")
            results.append((result, path))
    return results

def summarize(results: list[SearchResult]) -> None:
    for i, r in enumerate(results, 1):
        print(f"[{r.algorithm} #{i}] init={r.initial_objective:.2f} "
              f"final={r.final_objective:.2f} value={total_value(r.final_state)} "
              f"valid={is_valid(r.final_state)} objective={r.objective_mode} "
              f"iter={r.iterations} time={r.duration_s:.3f}s")
        e = r.extra
        if "iterations_per_restart" in e:
            print(f"    restart={e['restarts']} iterasi per
restart={e['iterations_per_restart']}")
```

```
        if "stuck_count" in e:
            print(f"    frekuensi stuck={e['stuck_count']}, "
                  f"{e['stuck_iterations']} iterasi ({e['stuck_iterations'] /
r.iterations:.0%})")
        if "population_size" in e:
            print(f"    populasi={e['population_size']} iterasi={r.iterations}")
        if "accepted_moves" in e:
            print(f"    move diterima={e['accepted_moves']}")
```

src/main.py

```
from __future__ import annotations

import argparse
from pathlib import Path

from algorithms import ALGORITHMS
from algorithms.base import SearchResult
from core.io_utils import load_problem
from core.objective import OBJECTIVE_MODES, set_objective_mode
from experiment import run_ga_experiment, run_repeated, summarize
from visualization.plots import plot_acceptance_probability, plot_history, plot_state


def parse_args() -> argparse.Namespace:
    p = argparse.ArgumentParser(description="Local search untuk container loading
(Mejiro Express)")
    p.add_argument("--input", required=True, help="Path file JSON problem")
    p.add_argument("--algo", choices=ALGORITHMS.keys())
    p.add_argument("--runs", type=int, default=3, help="Banyak run (spesifikasi: 3
kali)")
    p.add_argument("--seed", type=int, help="Seed agar hasil dapat direproduksi (run
ke-i memakai seed+i)")
    p.add_argument("--no-plot", action="store_true", help="Jangan buat plot sama
sekali")
    p.add_argument("--show", action="store_true",
                   help="Tampilkan plot secara interaktif (plot selalu disimpan ke
folder output)")
    p.add_argument("--ga-experiment", action="store_true",
                   help="Jalankan seluruh skenario eksperimen GA")
    p.add_argument("--objective", choices=OBJECTIVE_MODES, default="value",
                   help="Fungsi objektif: value = total value (default), "
                        "density = total value/berat tiap paket")
    # parameter algoritma
    p.add_argument("--max-sideways", type=int)
    p.add_argument("--max-restart", type=int)
    p.add_argument("--max-iterations", type=int)
    p.add_argument("--population", type=int)
    p.add_argument("--stuck-patience", type=int)
    p.add_argument("--initial-temp", type=float)
    p.add_argument("--min-temp", type=float)
    args = p.parse_args()
    if not args.algo and not args.ga_experiment:
        p.error("pilih --algo atau --ga-experiment")
    return args

def algo_params(args: argparse.Namespace) -> dict:
    mapping = {
        "sideways": {"max_sideways": args.max_sideways},
```

```
        "random_restart": {"max_restart": args.max_restart},
        "stochastic": {"max_iterations": args.max_iterations},
        "sa": {"max_iterations": args.max_iterations, "stuck_patience":
args.stuck_patience,
               "initial_temp": args.initial_temp, "min_temp": args.min_temp},
        "ga": {"population_size": args.population, "max_iterations":
args.max_iterations},
    }
    return {k: v for k, v in mapping.get(args.algo, {}).items() if v is not None}

def plot_result(r: SearchResult, json_path: Path, show: bool) -> None:
    """Simpan (dan opsional tampilkan) visualisasi satu run di samping file JSON
hasilnya."""
    base = json_path.with_suffix("")
    title = json_path.stem
    plot_state(r.initial_state, f"{title}: initial state", Path(f"{base}_initial.png"),
show)
    plot_state(r.final_state, f"{title}: final state", Path(f"{base}_final.png"), show)

    if "avg_history" in r.extra:
        plot_history(r.history, f"{title}: objective vs iterasi",
                     {"rata-rata populasi": r.extra["avg_history"]},
                     Path(f"{base}_history.png"), label="objective maksimum", show=show)
    elif "best_history" in r.extra:
        plot_history(r.history, f"{title}: objective vs iterasi",
                     {"best so far": r.extra["best_history"]},
                     Path(f"{base}_history.png"), show=show)
    else:
        plot_history(r.history, f"{title}: objective vs iterasi",
                     save_to=Path(f"{base}_history.png"), show=show)

    if "acceptance_probability" in r.extra:
        plot_acceptance_probability(r.extra["acceptance_probability"],
                                    save_to=Path(f"{base}_acceptance.png"), show=show)

def main() -> None:
    args = parse_args()
    set_objective_mode(args.objective)
    truck, packages = load_problem(args.input)

    if args.ga_experiment:
        runs = run_ga_experiment(truck, packages, n_runs=args.runs, seed=args.seed)
    else:
        runs = run_repeated(args.algo, truck, packages, n_runs=args.runs,
seed=args.seed,
                            **algo_params(args))
    summarize([r for r, _ in runs])

    if args.no_plot:
        return
    for r, path in runs:
        plot_result(r, path, args.show)
    print(f"Hasil dan plot disimpan di {runs[0][1].parent}")


if __name__ == "__main__":
    main()
```

2.10 Hasil Eksperimen

2.10.1 Lingkungan, Data Uji, dan Parameter

Eksperimen dijalankan dengan Python 3.11.9 pada sistem operasi Windows 11. Digunakan empat data uji dengan jumlah paket berbeda. Data 30, 40, dan 50 paket dibangkitkan acak dengan seed tetap (3170 + n); data 100 paket dibangkitkan dengan seed 3170. Ukuran paket 1–5 per sisi (tinggi maksimum 4), value 12–120, berat 3–40, dan sekitar 15% paket pecah-belah. Kapasitas truk dipilih sekitar 60% dari total berat paket (100 paket: sekitar 47%) sehingga tidak semua paket dapat dimuat.

Paket	Berkas	Truk	Kapasitas	Total berat	Total volume / volume truk	Fragile	Total value	UB
30	input_30_paket.json	10×20×5	400	667	656 / 1.000	4	2.025	1.613
40	input_40_paket.json	10×20×5	510	846	1.001 / 1.000	6	2.665	2.114
50	input_50_paket.json	10×20×5	690	1.143	1.323 / 1.000	6	3.282	2.760
100	input_100_paket.json	10×20×5	1.000	2.134	1.477 / 1.000	17	6.396	4.758

Tabel 2.10.1.1, Data Uji

Global optimum persoalan tidak diketahui. Sebagai pembanding, dihitung batas atas (UB) dengan menyelesaikan relaksasi persoalan berupa knapsack dua dimensi (total berat ≤ kapasitas dan total volume ≤ volume truk) menggunakan dynamic programming, dengan mengabaikan batasan geometri (tumpang-tindih, melayang, dan pecah-belah). Karena merupakan relaksasi, global optimum yang sebenarnya pasti ≤ UB, sehingga persentase terhadap UB (% UB) merupakan batas bawah dari kedekatan solusi terhadap global optimum.

Setiap algoritma dijalankan 3 kali pada setiap data uji dengan --seed 1: run ke-1, ke-2, dan ke-3 memakai seed 1, 2, dan 3, baik untuk state awal maupun untuk generator acak algoritma. Dengan demikian, run yang sama pada data yang sama memiliki state awal yang identik untuk semua algoritma, sehingga hasilnya dapat dibandingkan secara langsung.

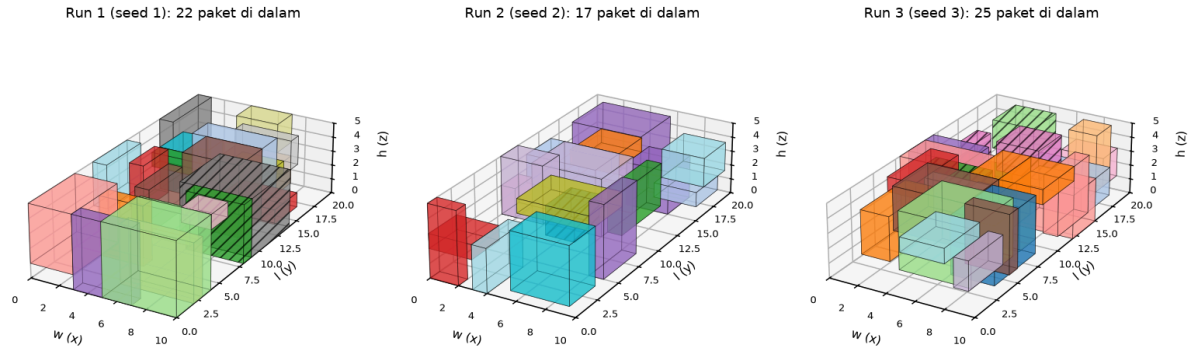
Adapun, parameter yang dipakai adalah:

Algoritma	Parameter
Steepest Ascent HC	-
HC with Sideways Move	max_sideways = 50
Random Restart HC	max_restart = 5 (100 paket: 3, karena durasi)
Stochastic HC	max_iterations = 10.000
Simulated Annealing	To = 1000, Tmin = 0,001, max_iterations = 5.000 (α ≈ 0,99724), stuck_patience = 100

Algoritma	Parameter
Genetic Algorithm	mutation_rate = 0,1, elitism = 1; populasi dan iterasi sesuai skenario pada Subbab 2.10.7

Tabel 2.10.1.2, Parameter algoritma

Tabel hasil memuat seluruh data uji. Dalam laporan ini, gambar state dan grafik kami tampilkan untuk data 50 paket saja, sedangkan untuk gambar untuk data lainnya tersedia di folder output/<n>_paket pada repository.



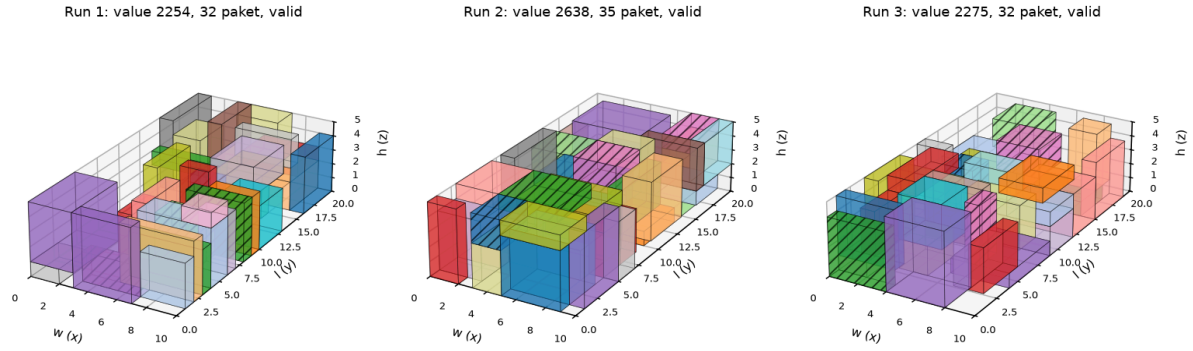
Gambar 2.10.1.1, State awal run 1–3, data 50 paket (sama untuk semua algoritma)

2.10.2 Steepest Ascent HC

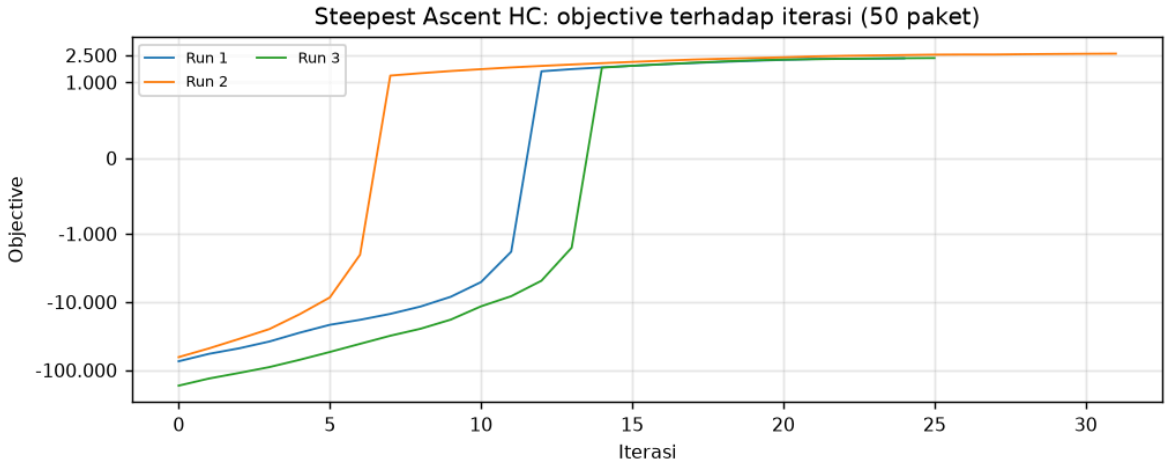
Paket	Run	Obj. awal	Total value akhir	% UB	Paket di dalam	Berat / kapasitas	Iterasi	Durasi (s)
30	1	-31.600	1.426	88,4%	18/30	395/400	13	0,36
	2	-17.582	1.501	93,1%	18/30	398/400	15	0,47
	3	-19.445	1.542	95,6%	18/30	398/400	12	0,34
40	1	-52.408	1.884	89,1%	24/40	510/510	19	1,01
	2	-62.872	1.889	89,4%	23/40	510/510	14	0,85
	3	-51.973	1.744	82,5%	23/40	506/510	14	1,00
50	1	-74.165	2.254	81,7%	32/50	689/690	24	2,60
	2	-64.476	2.638	95,6%	35/50	687/690	31	3,62
	3	-169.029	2.275	82,4%	32/50	688/690	25	3,34
100	1	-598.078	4.063	85,4%	50/100	1000/1000	39	32,46
	2	-418.918	4.284	90,0%	52/100	998/1000	34	25,96
	3	-2.344.300	4.317	90,7%	54/100	1000/1000	49	46,38

Tabel 2.10.2.1, Hasil Steepest Ascent HC (3 run per dataset)

Seluruh run Steepest Ascent menghasilkan state valid. Pada data 50 paket, total value akhir 2.254, 2.638, 2.275 (rata-rata 86,6% dari UB), dengan 24, 31, 25 iterasi dan durasi 2,60, 3,62, 3,34 detik. Paket yang tidak dimuat pada run 1: P08, P10, P12, P13, P14, P15, P20, P23, P26, P31, P32, P39, P40, P41, P42, P43, P48, P49.



Gambar 2.10.2.1, State akhir Steepest Ascent HC, data 50 paket, run 1–3



Gambar 2.10.2.2, Objective terhadap iterasi, Steepest Ascent HC, data 50 paket (sumbu y skala symlog)

2.10.3 HC with Sideways Move

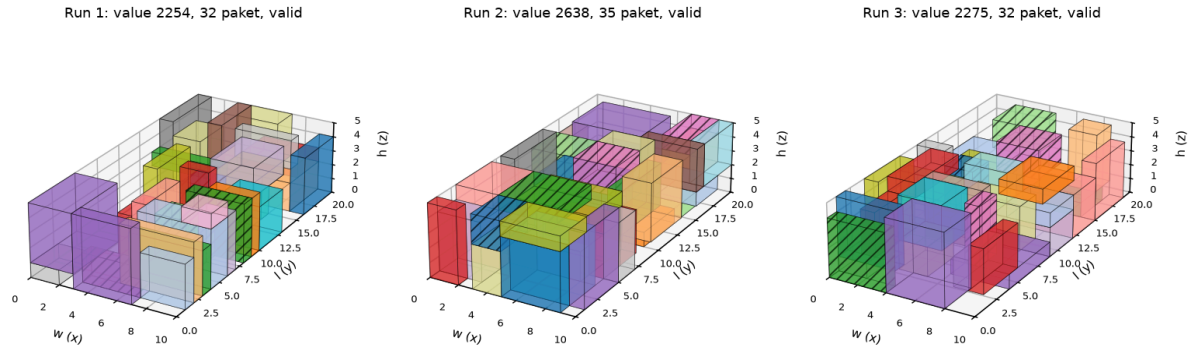
Parameter max_sideways = 50.

Paket	Run	Obj. awal	Total value akhir	% UB	Paket di dalam	Berat / kapasitas	Iterasi	Durasi (s)
30	1	-31.600	1.426	88,4%	18/30	395/400	63	2,23
	2	-17.582	1.501	93,1%	18/30	398/400	65	2,82
	3	-19.445	1.555	96,4%	18/30	399/400	64	2,53
40	1	-52.408	1.884	89,1%	24/40	510/510	69	5,46

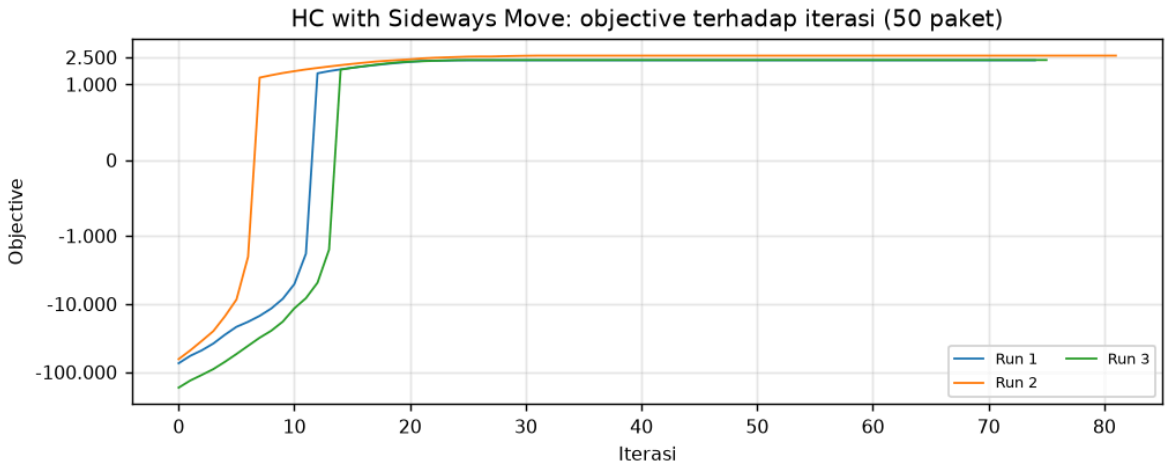
Paket	Run	Obj. awal	Total value akhir	% UB	Paket di dalam	Berat / kapasitas	Iterasi	Durasi (s)
	2	-62.872	1.889	89,4%	23/40	510/510	64	5,26
	3	-51.973	1.744	82,5%	23/40	506/510	64	4,81
50	1	-74.165	2.254	81,7%	32/50	689/690	74	10,55
	2	-64.476	2.638	95,6%	35/50	687/690	81	13,12
	3	-169.029	2.275	82,4%	32/50	688/690	75	12,98
100	1	-598.078	4.063	85,4%	50/100	1000/1000	89	76,44
	2	-418.918	4.284	90,0%	52/100	998/1000	84	74,24
	3	-2.344.300	4.317	90,7%	54/100	1000/1000	99	98,49

Tabel 2.10.3.1, Hasil HC with Sideways Move (3 run per dataset)

Pada semua data uji, total value akhir Sideways Move sama dengan Steepest Ascent pada seed yang sama, kecuali run 3 pada data 30 paket (1.555 dibanding 1.542). Banyak iterasinya selalu sekitar 50 lebih banyak daripada Steepest Ascent karena seluruh jatah sideways move terpakai.



Gambar 2.10.3.1, State akhir HC with Sideways Move, data 50 paket, run 1–3



Gambar 2.10.3.2, Objective terhadap iterasi, HC with Sideways Move, data 50 paket (sumbu y skala symlog)

2.10.4 Random Restart HC

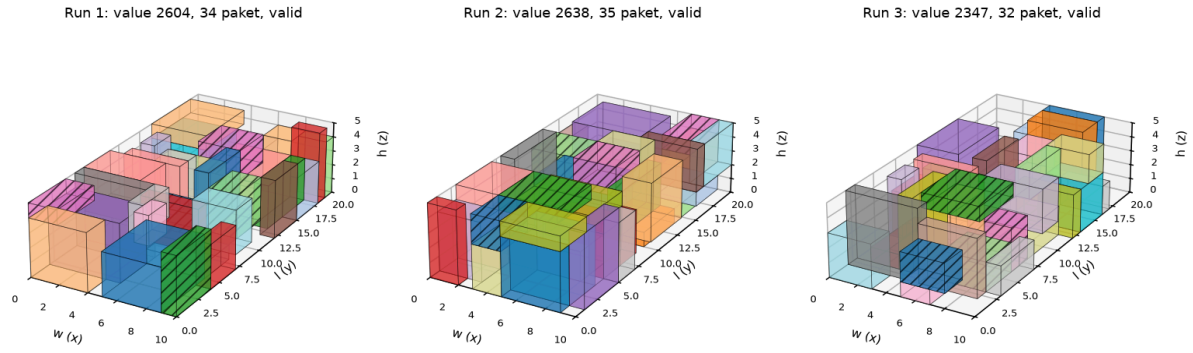
Parameter max_restart = 5 untuk data 30–50 paket dan 3 untuk data 100 paket. Kolom terakhir menunjukkan banyak iterasi pada setiap percobaan (percobaan pertama dimulai dari state awal run, sisanya dari state awal acak baru).

Paket	Run	Obj. awal	Total value akhir	% UB	Paket di dalam	Berat / kapasitas	Iterasi	Durasi (s)	Iterasi per restart
30	1	-31.600	1.591	98,6%	20/30	393/400	89	3,15	13, 13, 17, 17, 13, 16
	2	-17.582	1.560	96,7%	20/30	400/400	89	3,21	15, 15, 12, 16, 16, 15
	3	-19.445	1.542	95,6%	18/30	398/400	83	3,19	12, 12, 15, 16, 12, 16
40	1	-52.408	1.939	91,7%	24/40	507/510	108	6,87	19, 19, 15, 19, 19, 17
	2	-62.872	1.941	91,8%	23/40	508/510	104	7,42	14, 14, 20, 18, 20, 18
	3	-51.973	1.957	92,6%	25/40	508/510	106	7,65	14, 14, 21, 19, 22, 16
50	1	-74.165	2.604	94,3%	34/50	686/690	169	22,71	24, 24, 36, 27, 28, 30
	2	-64.476	2.638	95,6%	35/50	687/690	172	22,52	31, 31, 25, 29, 29, 27
	3	-169.029	2.347	85,0%	32/50	690/690	157	21,29	25, 25, 27, 24, 29, 27
100	1	-598.078	4.282	90,0%	53/100	1000/1000	166	133,32	39, 39, 44, 44

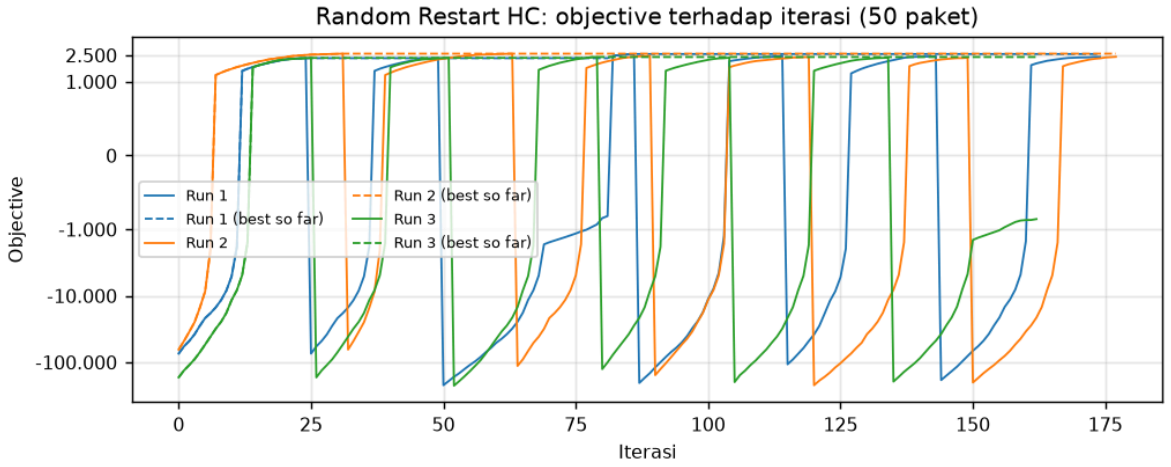
Paket	Run	Obj. awal	Total value akhir	% UB	Paket di dalam	Berat / kapasitas	Iterasi	Durasi (s)	Iterasi per restart
	2	-418.918	4.293	90,2%	55/100	998/1000	168	137,93	34, 34, 48, 52
	3	-2.344.300	4.317	90,7%	54/100	1000/1000	186	166,75	49, 49, 44, 44

Tabel 2.10.4.1, Hasil Random Restart HC (3 run per dataset)

Garis putus-putus pada grafik objective menunjukkan objective terbaik sejauh ini. Setiap penurunan tajam ke nilai sangat negatif adalah awal restart dari state acak baru.



Gambar 2.10.4.1, State akhir Random Restart HC, data 50 paket, run 1–3



Gambar 2.10.4.2, Objective terhadap iterasi, Random Restart HC, data 50 paket (sumbu y skala symlog)

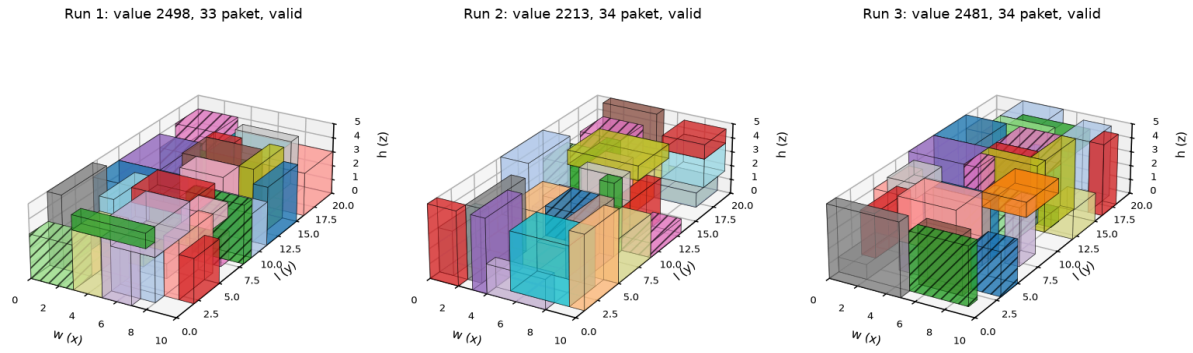
2.10.5 Stochastic HC

Parameter max_iterations = 10.000. Kolom terakhir menunjukkan banyak move yang diterima (move yang memperbaiki objektif).

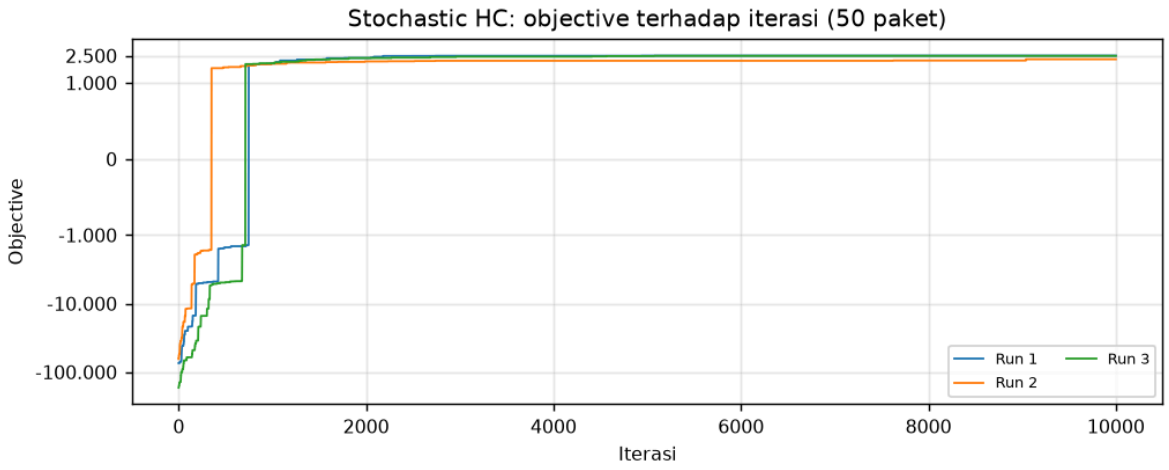
Paket	Run	Obj. awal	Total value akhir	% UB	Paket di dalam	Berat / kapasitas	Iterasi	Durasi (s)	Move diterima
30	1	-31.600	tidak valid (-484)	-	20/30	400/400	10.000	0,56	34
	2	-17.582	1.480	91,8%	18/30	397/400	10.000	0,51	29
	3	-19.445	1.432	88,8%	18/30	398/400	10.000	0,50	26
40	1	-52.408	1.734	82,0%	23/40	510/510	10.000	0,61	51
	2	-62.872	1.959	92,7%	25/40	509/510	10.000	0,67	50
	3	-51.973	1.893	89,5%	24/40	507/510	10.000	0,64	39
50	1	-74.165	2.498	90,5%	33/50	690/690	10.000	0,84	56
	2	-64.476	2.213	80,2%	34/50	683/690	10.000	0,84	51
	3	-169.029	2.481	89,9%	34/50	690/690	10.000	0,87	75
100	1	-598.078	3.918	82,3%	50/100	999/1000	10.000	1,61	113
	2	-418.918	tidak valid (-21.722)	-	51/100	997/1000	10.000	1,56	85
	3	-2.344.300	3.437	72,2%	50/100	1000/1000	10.000	1,80	128

Tabel 2.10.5.1, Hasil Stochastic HC (3 run per dataset)

Dari 10.000 iterasi, hanya puluhan hingga ratusan move yang diterima. State akhir tidak valid terjadi pada 30 paket run 1 dan 100 paket run 2.



Gambar 2.10.5.1, State akhir Stochastic HC, data 50 paket, run 1–3



Gambar 2.10.5.2, Objective terhadap iterasi, Stochastic HC, data 50 paket (sumbu y skala symlog)

2.10.6 Simulated Annealing

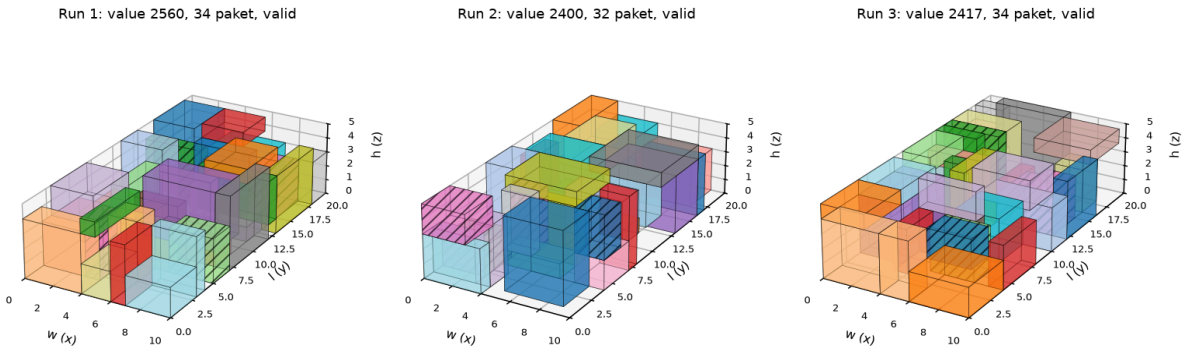
Parameter To = 1000, Tmin = 0,001, max_iterations = 5.000 (α ≈ 0,99724), dan stuck_patience = 100. Kolom terakhir adalah frekuensi stuck di local optima, diikuti banyak iterasi yang dihabiskan dalam kondisi stuck.

Paket	Run	Obj. awal	Total value akhir	% UB	Paket di dalam	Berat / kapasitas	Iterasi	Durasi (s)	Stuck (kali / iterasi)
30	1	-31.600	1.561	96,8%	20/30	398/400	5.000	0,97	10 / 2.400
	2	-17.582	1.578	97,8%	19/30	399/400	5.000	0,93	5 / 2.666
	3	-19.445	1.589	98,5%	20/30	397/400	5.000	0,97	6 / 2.874
40	1	-52.408	1.605	75,9%	20/40	507/510	5.000	0,95	4 / 2.830
	2	-62.872	1.798	85,1%	23/40	505/510	5.000	1,17	11 / 2.156
	3	-51.973	1.928	91,2%	22/40	507/510	5.000	1,08	7 / 2.698
50	1	-74.165	2.560	92,8%	34/50	687/690	5.000	1,39	4 / 2.269
	2	-64.476	2.400	87,0%	32/50	658/690	5.000	1,32	7 / 2.587
	3	-169.029	2.417	87,6%	34/50	679/690	5.000	1,62	7 / 2.227
100	1	-598.078	3.686	77,5%	47/100	980/1000	5.000	3,04	12 / 1.402
	2	-418.918	4.024	84,6%	51/100	996/1000	5.000	3,51	3 / 2.321
	3	-2.344.300	3.906	82,1%	48/100	992/1000	5.000	3,98	12 / 1.622

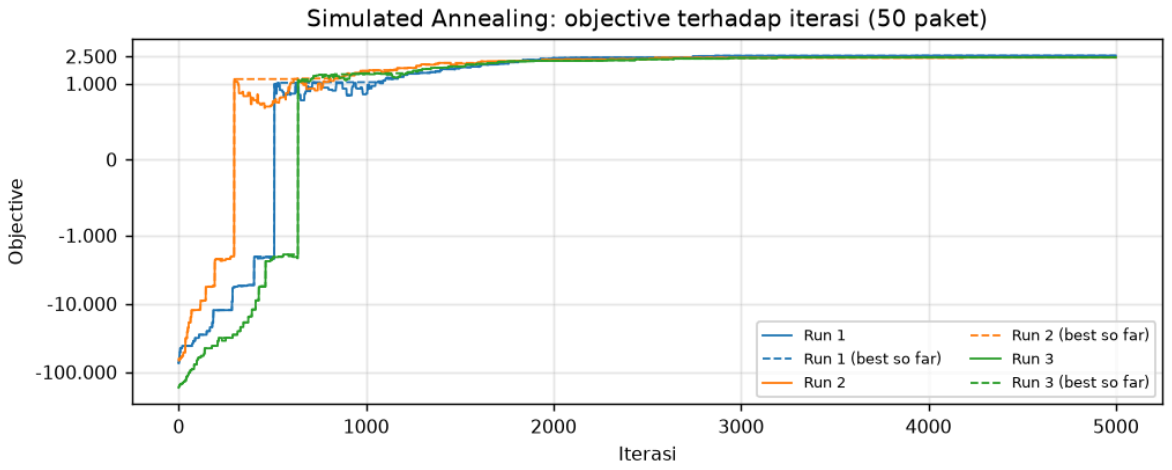
Tabel 2.10.6.1, Hasil Simulated Annealing (3 run per dataset)

Paket	Iterasi pertama state valid (run 1 / 2 / 3)	Temperatur saat itu	Rata-rata e^(ΔE/T) untuk move yang memperburuk
30	92 / 162 / 319	775,5 / 639,1 / 414,2	0,024 / 0,027 / 0,020
40	427 / 337 / 299	307,3 / 394,1 / 437,7	0,020 / 0,021 / 0,018
50	511 / 297 / 637	243,7 / 440,1 / 172,0	0,016 / 0,015 / 0,014
100	3.447 / 968 / 3.958	0,073 / 68,9 / 0,018	0,012 / 0,016 / 0,011

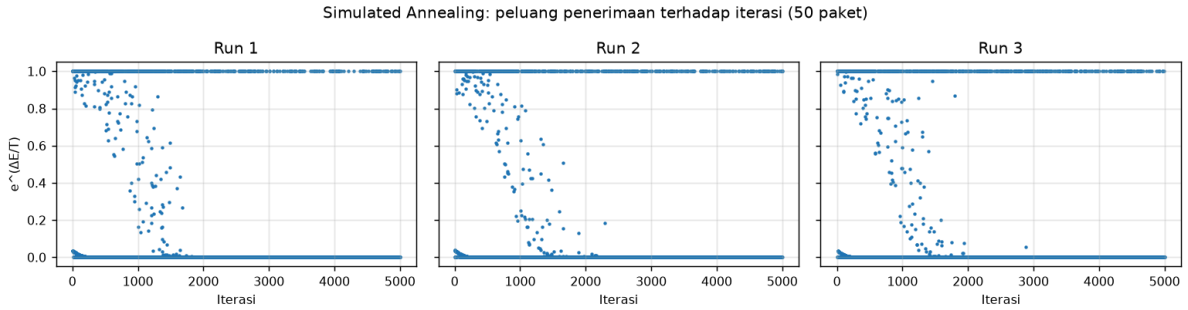
Tabel 2.10.6.2, Perilaku Simulated Annealing



Gambar 2.10.6.1, State akhir Simulated Annealing, data 50 paket, run 1–3



Gambar 2.10.6.2, Objective terhadap iterasi, Simulated Annealing, data 50 paket (sumbu y skala symlog)



Gambar 2.10.6.3, Plot e^(ΔE/T) terhadap iterasi, Simulated Annealing, data 50 paket

Pada plot e^(ΔE/T), titik bernilai 1 adalah move yang tidak memperburuk objektif. Titik di dekat 0 pada awal pencarian berasal dari move yang menambah pelanggaran (ΔE berorde ribuan, jauh lebih besar daripada T), sedangkan titik di antara 0 dan 1 berasal dari move yang hanya sedikit menurunkan value.

2.10.7 Genetic Algorithm

GA dijalankan dengan dua skenario. Pada skenario A, jumlah populasi dijadikan kontrol (50) dan banyak iterasi divariasikan (50, 100, 200). Pada skenario B, banyak iterasi dijadikan kontrol (100) dan jumlah populasi divariasikan (20, 50, 100). Konfigurasi populasi 50 dan iterasi 100 ada di kedua skenario sehingga hanya dijalankan sekali. Setiap konfigurasi dijalankan 3 kali pada setiap data uji. Tabel berikut menampilkan total value akhir.

Skenario A: populasi 50, variasi banyak iterasi

Paket	Iterasi	Run 1	Run 2	Run 3	Rata-rata durasi (s)
30	50	787	834	843	0,48
	100	1.147	1.062	1.104	1,19
	200	1.409	1.363	1.357	3,38
40	50	662	888	742	0,76
	100	873	1.049	1.045	1,43
	200	1.248	1.374	1.378	3,56
50	50	776	tidak valid (-2.566)	906	1,04
	100	1.012	1.067	1.250	1,86
	200	1.333	1.591	1.467	4,58
100	50	tidak valid (-87.454)	tidak valid (-30.795)	tidak valid (-30.460)	5,13
	100	tidak valid (-23.856)	tidak valid (-17.914)	tidak valid (-11.311)	7,84
	200	tidak valid (-4.345)	1.551	1.781	26,88

Tabel 2.10.7.1, Hasil GA skenario A (total value akhir)

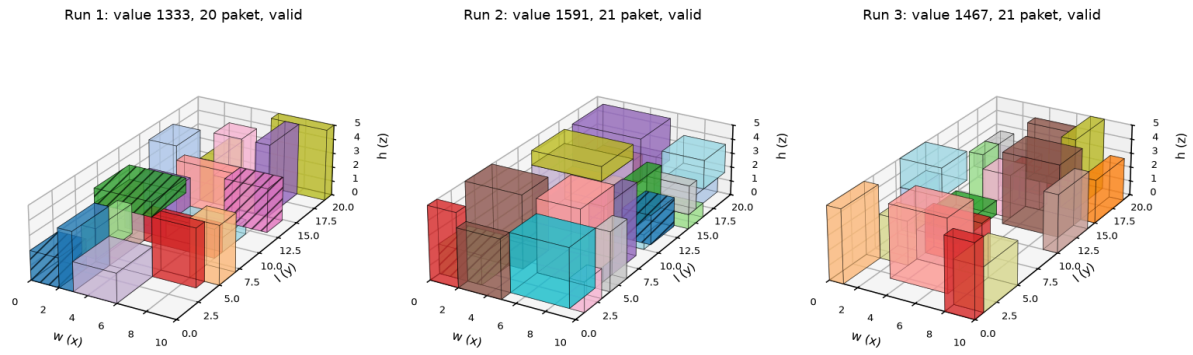
Skenario B: iterasi 100, variasi jumlah populasi

Paket	Populasi	Run 1	Run 2	Run 3	Rata-rata durasi (s)
30	20	929	1.298	1.172	0,51
	50	1.147	1.062	1.104	1,19
	100	1.025	1.012	1.181	1,92
40	20	1.354	670	1.157	0,63
	50	873	1.049	1.045	1,43
	100	1.233	1.013	1.110	2,79
50	20	tidak valid (-5.808)	tidak valid (-8.578)	tidak valid (-2.383)	1,03
	50	1.012	1.067	1.250	1,86
	100	1.099	1.285	828	3,81
100	20	tidak valid (-55.608)	tidak valid (-62.043)	tidak valid (-87.575)	16,25
	50	tidak valid (-23.856)	tidak valid (-17.914)	tidak valid (-11.311)	7,84
	100	1.284	tidak valid (-5.129)	826	33,45

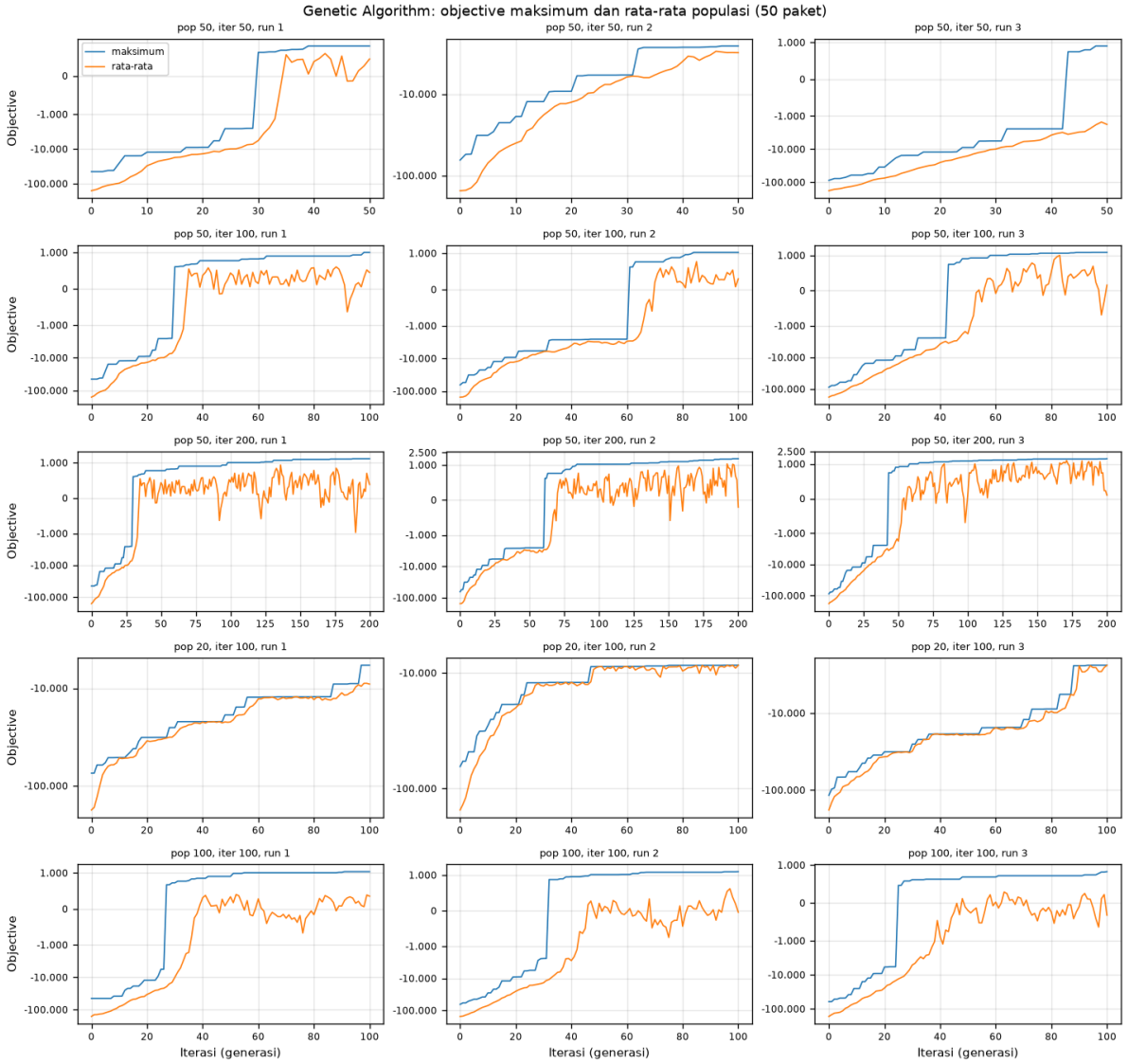
Tabel 2.10.7.2, Hasil GA skenario B (total value akhir)

Paket	pop 50, iter 50	pop 50, iter 100	pop 50, iter 200	pop 20, iter 100	pop 100, iter 100
30	13 / 14 / 19	13 / 14 / 19	13 / 14 / 19	41 / 33 / 29	14 / 16 / 13
40	17 / 31 / 40	17 / 31 / 40	17 / 31 / 40	34 / 67 / 57	21 / 24 / 21
50	30 / – / 43	30 / 61 / 43	30 / 61 / 43	– / – / –	27 / 32 / 25
100	– / – / –	– / – / –	– / 185 / 132	– / – / –	56 / – / 96

Tabel 2.10.7.3, Generasi pertama ditemukannya individu valid (run 1 / 2 / 3; – berarti tidak pernah)



Gambar 2.10.7.1, State akhir GA (populasi 50, iterasi 200), data 50 paket, run 1–3



Gambar 14 Objective maksimum dan rata-rata populasi terhadap iterasi untuk setiap konfigurasi GA, data 50 paket (sumbu y skala symlog)

2.10.8 Eksperimen Tambahan

Steepest Ascent dan Simulated Annealing dijalankan dengan mode objective value dan density pada data 30, 40, dan 50 paket. Tabel berikut menampilkan total value state akhirnya.

Paket	Algoritma	Mode value	Mode density
30	Steepest Ascent	1.426 / 1.501 / 1.542	1.382 / 1.554 / 1.585
	SA	1.561 / 1.578 / 1.589	1.547 / 1.520 / 1.584
40	Steepest Ascent	1.884 / 1.889 / 1.744	1.922 / 2.059 / 1.836

Paket	Algoritma	Mode value	Mode density
	SA	1.605 / 1.798 / 1.928	1.713 / 1.897 / 1.810
50	Steepest Ascent	2.254 / 2.638 / 2.275	2.384 / 2.539 / 2.232
	SA	2.560 / 2.400 / 2.417	2.128 / 2.143 / 2.209

Tabel 18 Total value akhir dengan mode objective value dan density (run 1 / 2 / 3)

2.11 Analisis Hasil Eksperimen

2.11.1 Kedekatan terhadap Global Optimum

Global optimum tidak diketahui secara pasti karena permasalahan bersifat NP-hard. Oleh karena itu, kualitas solusi dievaluasi menggunakan upper bound yang dihitung melalui persoalan knapsack dua dimensi.

Algoritma	30 Paket	40 Paket	50 Paket	100 Paket
Steepest Ascent HC	92,4%	87,0%	86,6%	88,7%
HC with Sideways Move	92,6%	87,0%	86,6%	88,7%
Random Restart HC	97,0%	92,0%	91,7%	90,3%
Stochastic HC	90,3% (2/3 valid)	88,1%	86,9%	77,3% (2/3 valid)
Simulated Annealing	97,7%	84,1%	89,1%	81,4%
GA (pop 50, iter 200)	85,3%	63,1%	53,0%	35,0% (2/3 valid)

Tabel 19 Rata-rata total value akhir (run valid) sebagai persentase dari UB

Hasil terbaik pada setiap data uji adalah 1.591 (98,6% dari UB, 30 paket); 1.959 (92,7% dari UB, 40 paket); 2.638 (95,6% dari UB, 50 paket); 4.317 (90,7% dari UB, 100 paket). Karena UB mengabaikan batasan geometri, jarak sebenarnya terhadap global optimum lebih kecil daripada selisih terhadap UB. Rata-rata Random Restart mencapai 90–97% dari UB, sehingga hasilnya cukup dekat dengan global optimum. Persentase Random Restart menurun seiring bertambahnya jumlah paket (97,0% menjadi 90,3%) karena ruang pencarian bertambah besar dan geometri menjadi lebih ketat: total volume paket pada data 40, 50, dan 100 paket melebihi volume truk, sehingga UB (yang hanya membatasi total volume) makin longgar dibanding solusi yang benar-benar dapat disusun.

Semua algoritma tetap berhenti di local optimum. Neighbor yang tersedia hanya mengubah satu paket (move/rotate) atau menukar posisi dua paket. Untuk mencapai kombinasi paket yang lebih baik, sering kali diperlukan mengeluarkan satu paket lalu memasukkan paket lain yang lebih bernilai di tempat lain. Langkah pertama menurunkan objektif sehingga tidak dipilih oleh Hill-climbing. Swap antara paket di dalam dan di luar truk dapat

melakukan pertukaran ini dalam satu langkah, tetapi hanya jika paket pengganti muat pada posisi dan orientasi yang sama.

2.11.2 Pola Pencarian

Grafik objective semua algoritma memperlihatkan dua fase. Pada fase pertama, objective naik dalam lompatan besar karena setiap langkah menghilangkan pelanggaran (kelipatan M, yaitu total value seluruh paket + 1), biasanya dengan mengeluarkan paket yang bertumpuk atau melayang. Setelah objective menjadi positif (state valid), kenaikan menjadi kecil karena setiap langkah hanya menambah satu paket atau menukar paket dengan paket yang lebih bernilai hingga kapasitas berat hampir penuh. Pada data 50 paket, Steepest Ascent mencapai state valid dalam 7–14 iterasi, lalu memerlukan 11–24 iterasi lagi untuk mengisi truk.

2.11.3 Perbandingan Varian Hill-climbing

Sideways Move. Sideways Move hampir tidak pernah memperbaiki hasil Steepest Ascent, tetapi durasinya 2–7 kali lebih lama karena menghabiskan seluruh jatah 50 sideways move. Plateau di sekitar local optimum terdiri atas state dengan subset paket yang sama dan hanya berbeda posisi atau orientasi, sehingga value tidak berubah. Karena pemilihan neighbor terbaik bersifat deterministik dan tidak ada larangan kembali ke state sebelumnya, algoritma dapat bergerak bolak-balik di antara state-state plateau yang sama.

Random Restart. Random Restart memberikan rata-rata terbaik di antara varian Hill-climbing pada semua data uji, karena mengambil yang terbaik dari 6 (atau 4) local optimum. Contohnya pada data 50 paket, run 1 naik dari 2.254 (Steepest Ascent) menjadi 2.604. Biayanya sebanding dengan banyak percobaan: durasinya 4–8 kali Steepest Ascent. Banyak iterasi tiap percobaan relatif seragam (misalnya 24–36 pada data 50 paket), sehingga durasi Random Restart dapat diperkirakan dari banyak restart.

Stochastic. Stochastic Hill-climbing jauh lebih cepat (kurang dari 2 detik bahkan untuk 100 paket) karena hanya mengevaluasi satu neighbor per iterasi, dan hasilnya kompetitif pada data 40–50 paket. Namun, hasilnya kurang konsisten: 30 paket run 1 dan 100 paket run 2 berakhir tidak valid. Pada kedua kasus, sisa pelanggaran hanya dapat dihilangkan oleh sedikit move spesifik, dan pemilihan acak tidak menemukannya dalam 10.000 iterasi. Steepest Ascent tidak memiliki masalah ini karena selalu memeriksa seluruh neighbor.

2.11.4 Simulated Annealing

Dengan parameter bawaan (5.000 iterasi), seluruh run SA berakhir valid. SA memberikan rata-rata terbaik dari semua algoritma pada data 30 paket (97,7% dari UB) dan berada di

antara Steepest Ascent dan Random Restart pada data 50 paket (89,1%). Namun, SA tertinggal dari Steepest Ascent pada data 40 paket (84,1%, terutama karena run 1 hanya mencapai 1.605) dan 100 paket (81,4%). Ada dua faktor yang membatasi SA.

1) Anggaran evaluasi jauh lebih kecil. SA hanya mengevaluasi 5.000 neighbor per run, sedangkan Steepest Ascent pada data 100 paket mengevaluasi sekitar 15.301 neighbor per iterasi × 34–49 iterasi, atau ratusan ribu evaluasi. Durasi SA (3,5 detik pada 100 paket) juga jauh lebih singkat daripada Steepest Ascent (34,9 detik).

2) Skala temperatur tidak sesuai dengan skala penalti. Selama state belum valid, setiap move yang menambah pelanggaran memiliki ΔE berorde M (ribuan), sehingga e^(ΔE/T) ≈ 0 bahkan pada T = 1000. Akibatnya, kualitas akhir SA sangat bergantung pada seberapa cepat state menjadi valid. Pada data 30–50 paket, SA sudah valid pada iterasi 92–637, saat T masih sekitar 172–776, sehingga masih tersisa ribuan iterasi untuk berpindah di antara state valid, termasuk menerima move yang sedikit menurunkan value. Pada data 100 paket, SA baru valid pada iterasi 3.447, 968, 3.958, saat T tinggal 0,073, 68,9, 0,018. Run yang paling cepat valid (run 2) juga memberikan hasil terbaik (4.024). Pada plot e^(ΔE/T) data 50 paket, titik dengan nilai jelas di antara 0 dan 1 (0,05–0,95) terakhir muncul pada iterasi 1.674–2.883, saat T ≈ 0,35–9,8. Setelah itu, move yang menurunkan value hampir tidak pernah diterima, sehingga sisa pencarian berperilaku seperti Stochastic Hill-climbing. Hal ini sesuai dengan banyaknya iterasi yang dihabiskan dalam kondisi stuck (28–57% iterasi, dengan frekuensi stuck 3–12 kali per run).

Eksperimen tambahan (Tabel 17) mendukung analisis ini. Dengan 27.625 iterasi (α ≈ 0,9995), rata-rata hasil SA naik dari 2.459 menjadi 2.534 pada data 50 paket dan dari 3.872 menjadi 4.264 pada data 100 paket. Hasil ini lebih baik daripada rata-rata Steepest Ascent (2.389 dan 4.221) dan mendekati Random Restart (2.530 dan 4.297). Biayanya adalah durasi yang naik sekitar 5–6 kali (8,2 dan 18,6 detik). Durasi ini sekitar 2,6 kali Steepest Ascent pada data 50 paket (3,2 detik), tetapi masih lebih singkat daripada Steepest Ascent pada data 100 paket (34,9 detik).

2.11.5 Genetic Algorithm

Pengaruh banyak iterasi. Pada skenario A, hasil GA selalu naik ketika banyak iterasi ditambah. Karena seed sama, run dengan 200 iterasi adalah lanjutan persis dari run dengan 100 dan 50 iterasi (generasi pertama ditemukannya individu valid identik pada Tabel 16), sehingga skenario ini memperlihatkan secara langsung bahwa GA masih terus membaik pada iterasi ke-200. Misalnya pada data 50 paket, rata-rata naik dari 841 (iterasi

50) menjadi 1.110 (iterasi 100) dan 1.464 (iterasi 200). Durasinya bertambah linear terhadap banyak iterasi.

Pengaruh jumlah populasi. Pada skenario B, populasi 20 tidak pernah menghasilkan individu valid pada data 50 dan 100 paket, karena keragaman populasi terlalu kecil untuk menghilangkan pelanggaran hanya dengan crossover dan mutasi. Populasi yang lebih besar mempercepat ditemukannya individu valid (data 50 paket: generasi 25–32 untuk populasi 100, dibanding 30–61 untuk populasi 50), tetapi tidak selalu memberi hasil akhir lebih baik dalam 100 iterasi, sementara durasinya sekitar 2 kali lipat. Pada data 30–40 paket, populasi 20 bahkan kadang lebih baik karena seleksi lebih cepat terfokus pada individu terbaik.

Mengapa GA tertinggal. Hasil GA terbaik (53,0% dari UB pada data 50 paket) jauh di bawah Hill-climbing dan SA. Penyebabnya terletak pada representasi dan operatornya:

- Crossover merusak. Anak mengambil posisi sebagian paket dari induk pertama dan sisanya dari induk kedua. Posisi tersebut hanya bermakna dalam susunan induknya masing-masing, sehingga hasil penggabungan hampir selalu menimbulkan tumpang-tindih dan paket melayang, bahkan jika kedua induk valid.
- Seleksi tidak membedakan individu valid. Bobot roulette f − f_min + 1 didominasi oleh penalti (puluhan ribu), sehingga perbedaan value antar-individu valid (puluhan hingga ratusan) hampir tidak berpengaruh pada peluang terpilih.
- Mutasi lemah. Dengan mutation_rate 0,1, sekitar 90% anak tidak dimutasi sama sekali, padahal mutasi adalah satu-satunya operator yang dapat memperbaiki tumpang-tindih hasil crossover.

Akibatnya, rata-rata objective populasi berfluktuasi tajam di sekitar nol setelah individu valid ditemukan (Gambar 14), dan kemajuan objective maksimum terutama berasal dari elitisme ditambah mutasi kecil pada individu terbaik.

2.11.6 Durasi dan Konsistensi

Algoritma	30 Paket	40 Paket	50 Paket	100 Paket
Steepest Ascent HC	0,39	0,95	3,19	34,93
HC with Sideways Move	2,53	5,18	12,21	83,06
Random Restart HC	3,18	7,31	22,17	146,00
Stochastic HC	0,52	0,64	0,85	1,66
Simulated Annealing	0,95	1,07	1,45	3,51

Algoritma	30 Paket	40 Paket	50 Paket	100 Paket
GA (pop 50, iter 200)	3,38	3,56	4,58	26,88

Tabel 20 Rata-rata durasi per run (detik)

Durasi Hill-climbing yang mengevaluasi seluruh neighbor (Steepest Ascent, Sideways, Random Restart) tumbuh paling cepat terhadap jumlah paket: Steepest Ascent naik dari 0,39 detik (30 paket) menjadi 34,9 detik (100 paket). Satu iterasi mengevaluasi $O(n^2)$ swap, 3n rotasi, dan $O(n^2)$ kandidat move, masing-masing dalam O(n) berkat evaluasi inkremental, dan banyak iterasi juga bertambah seiring jumlah paket. Stochastic HC dan SA memiliki banyak iterasi tetap sehingga durasinya hanya tumbuh linear terhadap biaya satu evaluasi.
Dari segi konsistensi, Steepest Ascent dan Sideways Move menghasilkan simpangan baku value akhir yang cukup besar antar-run (misalnya 176 pada data 50 paket), karena setiap state awal mengarah ke local optimum yang berbeda. Random Restart paling konsisten (8 pada data 40 paket) karena memilih yang terbaik dari beberapa local optimum. GA paling tidak konsisten, termasuk run-run yang tidak valid.

2.11.7 Mode Objective Density

Pada eksperimen tambahan (Tabel 18), mode density membuat Steepest Ascent sedikit lebih baik pada data 40 paket (ketiga run) tetapi hasilnya bercampur pada data 30 dan 50 paket, sedangkan pada SA mode density menghasilkan total value lebih rendah pada data 30 dan 50 paket dan sedikit lebih tinggi pada data 40 paket. Mode density mendorong pemuatan paket ringan bernilai tinggi lebih dahulu, yang membantu ketika berat adalah batasan utama, tetapi dapat mengabaikan paket berat yang bernilai sangat besar. Karena tujuan persoalan adalah memaksimalkan total value, mode value tetap dipakai sebagai fungsi objektif utama, sedangkan density lebih tepat dipakai sebagai heuristik pembanding.

BAB III | Kesimpulan dan Saran

3.1 Kesimpulan

1. Persoalan pengisian peti kemas Mejiro Express dapat dimodelkan sebagai state berupa pemetaan setiap paket ke posisi (atau di luar truk) dan orientasinya, dengan fungsi objektif total value dikurangi penalti pelanggaran berbobot M > total value semua paket. Dengan bobot ini, state valid selalu lebih baik daripada state yang melanggar constraint.
2. Varian Hill-climbing memberikan hasil terbaik pada sebagian besar data uji. Random Restart paling konsisten (rata-rata 97,0%, 92,0%, 91,7%, 90,3% dari UB untuk 30, 40, 50, 100 paket), dengan durasi 4–8 kali Steepest Ascent. Sideways Move tidak memberi perbaikan berarti, sedangkan Stochastic HC sangat cepat tetapi kadang berakhir tidak valid.
3. Simulated Annealing dengan parameter bawaan (5.000 iterasi) berakhir valid pada semua run dan memberikan rata-rata terbaik pada data 30 paket, tetapi tertinggal dari Hill-climbing pada data 40 dan 100 paket karena jumlah evaluasinya kecil dan skala temperaturnya tidak sesuai dengan skala penalti. Dengan 27.625 iterasi ($\alpha \approx 0{,}9995$), SA lebih baik daripada Steepest Ascent dan mendekati Random Restart.
4. Genetic Algorithm memberikan hasil paling rendah. Menambah banyak iterasi selalu memperbaiki hasil, sedangkan populasi yang terlalu kecil (20) gagal menemukan state valid pada data 50 dan 100 paket. Keterbatasan utamanya adalah crossover yang merusak susunan paket dan seleksi yang didominasi penalti.

3.2 Saran

1. Memperbaiki GA dengan operator yang sadar kelayakan: crossover per paket disertai langkah perbaikan (repair) yang mengeluarkan paket yang bertabrakan, mutation rate yang lebih tinggi, dan seleksi berbasis peringkat (rank) atau turnamen agar perbedaan value antar-individu valid berpengaruh.

2. Menambahkan neighbor yang mengganti paket di dalam truk dengan paket di luar truk pada posisi yang tidak harus sama, karena keterbatasan utama Hill-climbing pada persoalan ini adalah pemilihan subset paket.
3. Menambahkan tabu list atau pemilihan acak di antara neighbor bernilai sama pada Sideways Move agar tidak berputar di plateau yang sama.
4. Membandingkan algoritma dengan anggaran evaluasi atau waktu yang setara agar perbandingan lebih adil.

Lampiran

Link Repository

https://github.com/rhenprojects/IF3170_G18_YuzukiAkatsuki

Lampiran Penggunaan AI

Artificial Intelligence (AI) digunakan sebagai alat bantu dalam penyusunan dan penyempurnaan redaksi laporan agar penyampaian informasi lebih jelas, sistematis, dan mudah dipahami. AI juga dimanfaatkan sebagai sarana diskusi pembelajaran untuk memperdalam pemahaman mengenai konsep optimasi pencarian (*local search*), perancangan fungsi objektif (*objective function*), penanganan *constraint*, karakteristik algoritma, serta analisis dan evaluasi kualitas solusi yang diperoleh. Penggunaan AI dilakukan sebagai pendukung proses belajar dan penalaran, sedangkan perancangan dan implementasi algoritma, pelaksanaan eksperimen, serta analisis akhir hasil pengujian tetap menjadi tanggung jawab anggota kelompok.

Pembagian Tugas Kelompok

NIM	Nama	Pembagian
13524012	Tengku Naufal Saqib	Hill Climbing
13524039	Rhenaldy Cahyadi Putra	Simulated Annealing
13524113	Fauzan Mohamad Abdul Gani	Genetic Algorithm

Referensi

[1] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by Simulated Annealing,” Science, vol. 220, no. 4598, pp. 671–680, May 1983, doi: 10.1126/science.220.4598.671.

[2] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[3] T. G. Crainic, G. Perboli, and R. Tadei, "Extreme Point-Based Heuristics for Three-Dimensional Bin Packing," INFORMS Journal on Computing, vol. 20, no. 3, pp. 368–384, 2008.